AN0218
Application Note

AT32 PMSM FOC Hall Quick Start Guide

Introduction

This document introduces how to use AT-MOTOR-EVB evaluation board equipped with AT32 motor control library to drive PMSM motor (with Hall sensors), and details debugging SOP.
ArteryTek offers many motor development boards, and this document uses the AT-MOTOR-EVB as an example for illustration.

Applicable products:

Product series	AT32F402/405 series
	AT32F403A/407 series
	AT32F413 series
	AT32F415 series
	AT32F421 series
	AT32F422/426 series
	AT32F425 series
	AT32F423 series
	AT32F435/437 series
	AT32F45x series
	AT32L021 series
	AT32M412/M416 series

Contents

List of tables

List of figures

1 Motor control library algorithms

This document contains the following motor control algorithms:

Target motor type: Three-phase permanent magnet synchronous motor (BLDC)
Control modes:

- FOC vector control

Three-phase PWM method:

- SVPWM

Phase current sensing methods:

- 3-shunt current sensing
- 2-shunt current sensing
- 1-shunt current sensing and reconstruction method

Rotor position detection mode:

- Hall effect position detection

Sensor-based FOC sinusoidal control mode:

- Voltage vector control
- Torque control (current vector control)
- Speed control
- Field weakening control
- Positioning control

Auto tuning functions:

- Hall phase sequence self-learning (for hall sensor only)
- Motor winding parameters identification
- Current PI control parameters self-tuning

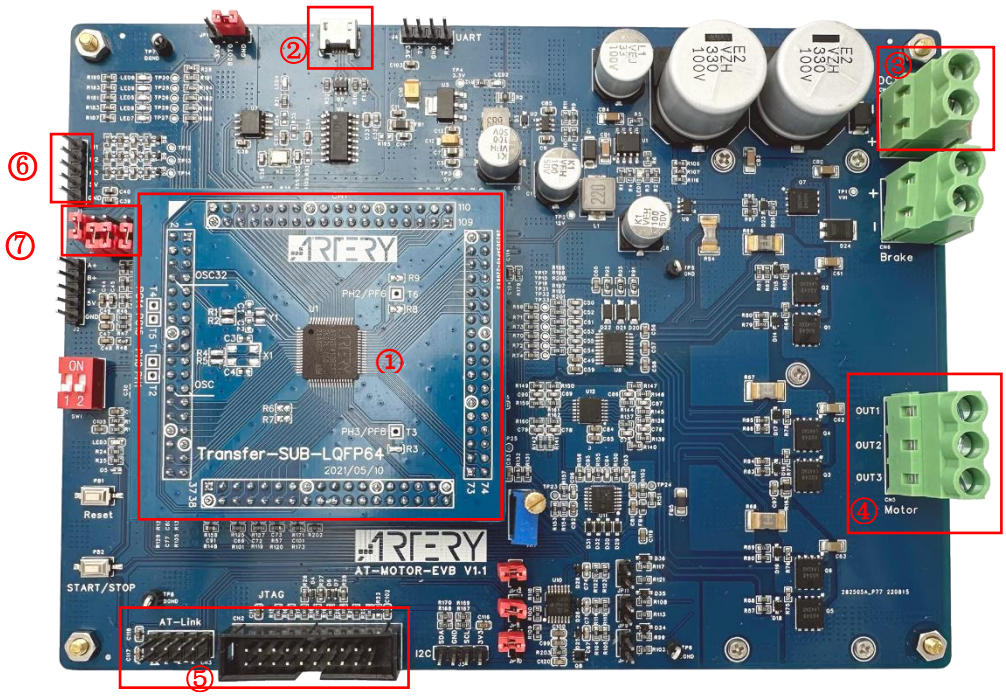
2 Environment requirements

2.1 Hardware requirements

Hardware resources required include a PMSM (BLDC) motor, AT-Link or third-party debugger and a motor control evaluation board (AT-MOTOR-EVB). For details about hardware configurations, please refer to *AT32_LV_Motor_Control_EVB_User _Manual* (UM0011 for EVB_V1.x or UM0014 for EVB_V2.x).

- PMSM (BLDC) motor
- Debugger
- Motor control EVB: AT-MOTOR-EVB V1.x or AT-MOTOR-EVB V2.x

Figure 1. AT-MOTOR-EVB V1.x evaluation board



① AT32 MCU transfer board (part number: AT32F413RCT7)

② USB type B-to- RS-232 connector: connected with PC via USB cable for fine-tuning motor control parameters

③ Power input connector CN1: connected to 24 V

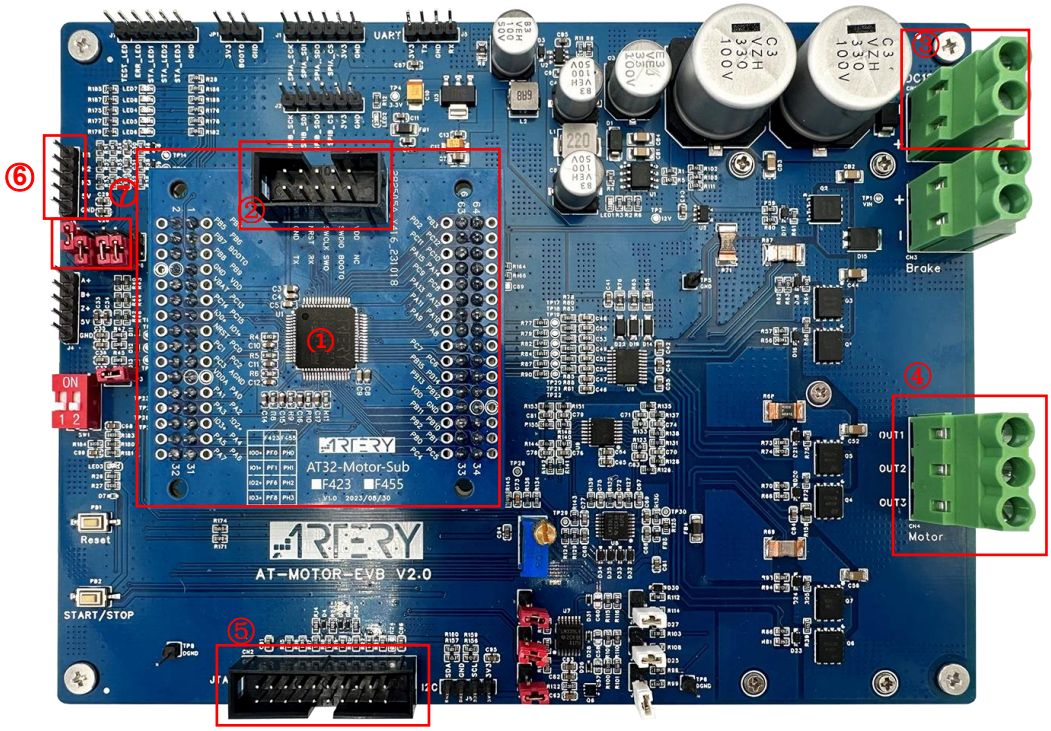
④ Driver output connector CN5: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively

⑤ AT-Link or J-Link connector: for debugging purposes

⑥ Hall sensor connector J1: connected with H1, H2, H3, 5V, GND

⑦ Jumper settings: JP2 CLOSED; JP3, JP4, JP5 and JP6 OPEN

Figure 2. AT-MOTOR-EVB V2.x



① AT32 MCU transfer board (part number: AT32F413RCT7)

② AT-Link connector: for debug and download purposes, and connecting with UI software for fine-tuning motor parameters

③ Power input connector CN1: connected to 24 V

④ Driver output connector CN4: connected to three-phase motor U, V and W (yellow, green and blue) leads, respectively

⑤ J-Link connector: for debugging purposes

⑥ Hall sensor connector J3: connected with H1, H2, H3, 5V, GND

⑦ Jumper settings: JP2 CLOSED; JP3, JP4, JP5 and JP6 OPEN

2.2 Software requirements

1) Taking AT32F413 as an example, open AT32F413_MC_Library_Project\motor_evb_2v0 \at32f413\pmsm_foc_hall_sensor (note [1])

2) Run “ArteryMotorMonitor.exe” (installation not required)

3) In Keil, go to ***Options - Read/Only Memory Areas*** to set ROM according to the Flash memory size of each AT32 MCU (see Table 1). For example: for AT32F413RCT7 with a 256KB Flash memory, its IROM1 start address is at 0x8000000 with a size of 0x3F800, whereas its IROM2 start address is at 0x803F800 with a size of 0x800 (see Figure 3).
In AT32IDE, modify *ld file according to the Flash memory size of each AT32 MCU (see Figure 4).
In IAR Systems, modify *icf file according to the Flash memory size of each AT32 MCU (see Figure 5).

Table 1. Flash memory size and ROM configuration

Flash size	**4032K**	**1024K**	**512K**	**448K**	**256K**
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	**128K**	**64K**	**32K**	**16K**
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

Note [1]: This demo does not support using of Keil complier version 5 and O0 optimization level for compiling purpose simultaneously. Thus either Keil compiler version 6 or O1 optimization level is used for this purpose. As AT32 BSP source code does not support V6.15 compiler, it is recommended to use Keil compiler version 5 and O1 optimization level.

Figure 3. AT32F413RCT7 ROM configuration (Keil)

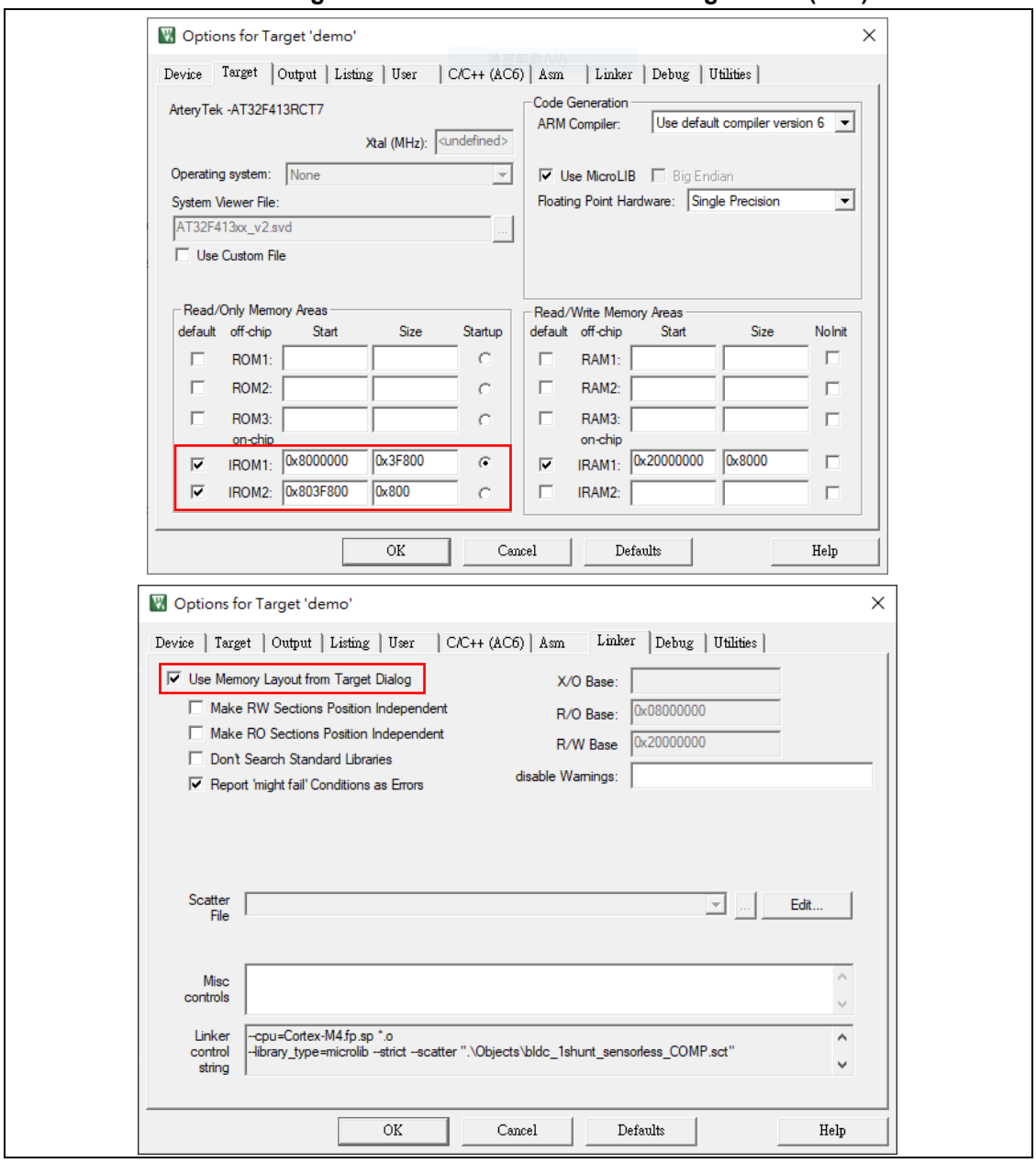
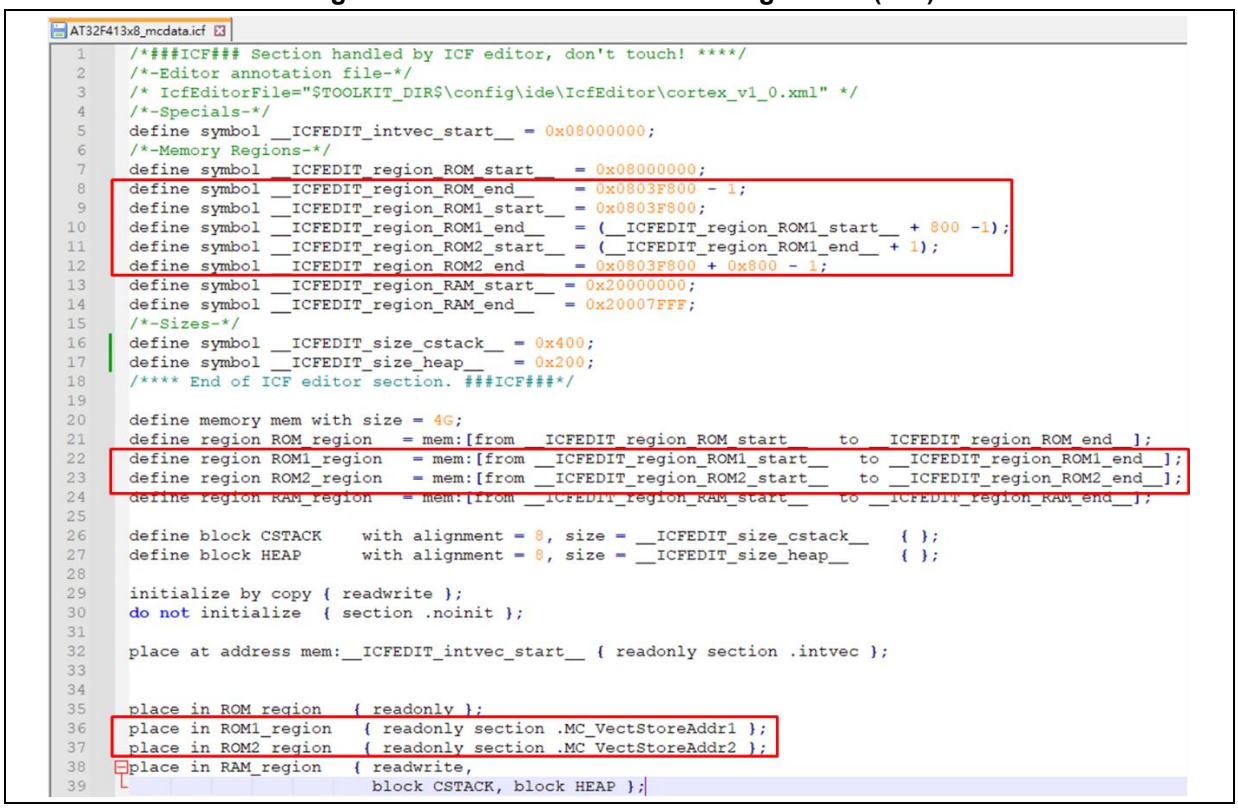


Figure 4. AT32F413RCT7 ROM configuration (AT32IDE)

AT32F413xC_FLASH.ld

```
_estack = 0x20008000;    /* end of RAM */

/* Generate a link error if heap and stack don't fit into RAM */
_Min_Heap_Size = 0x200;      /* required amount of heap  */
_Min_Stack_Size = 0x400; /* required amount of stack */

/* Specify the memory areas */
MEMORY
{
FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
MC_DATA (r)     : ORIGIN = 0x0803F800, LENGTH = 0x800
RAM (xrw)       : ORIGIN = 0x20000000, LENGTH = 32K
}

/* Define output sections */
SECTIONS
{
  /* The startup code goes first into FLASH */
  .isr_vector :
  {
    . = ALIGN(4);
    KEEP(*(.isr_vector)) /* Startup code */
    . = ALIGN(4);
  } >FLASH

  .mc_data :
  {
    . = ALIGN(4);
    . = ORIGIN(MC_DATA);
    _MC_VectStoreAddr1 = .;
    *(.MC_VectStoreAddr1);
    . = ALIGN(4);
    . = ORIGIN(MC_DATA) + 800;
    _MC_VectStoreAddr2 = .;
    *(.MC_VectStoreAddr2);
    . = ALIGN(4);
  } >MC_DATA

  /* The program code and other data goes into FLASH */
  .text :
  {
    . = ALIGN(4);
    *(.text)           /* .text sections (code) */
    *(.text*)          /* .text* sections (code) */
    *(.glue_7)         /* glue arm to thumb code */
    *(.glue_7t)        /* glue thumb to arm code */
    *(.eh_frame)
```

Figure 5. AT32F413RCT7 ROM configuration (IAR)



3 Quick start guide

Follow the procedures below to fine-tune a PMSM motor with hall sensor by using FOC control method.

Step 1: Modify motor control drive mode and parameters according to motor parameters and application requirements (see Section 3.1).

Step 2: Connect with UI software (see Section 3.2).

Step 3: Select open loop control mode to check whether motor is running normally (see Section 3.3).

Step 4: Enable hall phase sequence self-learning (see Section 3.4).

Step 5: Verify if the motor can operate based on the Hall sensor position using voltage control (see Section3.5).

Step 6: Current control

- Motor winding parameter identification (see Section 3.6)
- Current PI controller parameter auto identification (see Section 3.6)
- ID tune: fine-tuning Kp and Ki values of current loop (see Section 3.7)
- IQ tune: fine-tuning Kp and Ki values of current loop (see Section 3.8)
- Current loop control (see Section 3.9)

Step 7: Tune speed control parameters (see section 3.10).

Step 8: Tune low-speed voltage control parameters if there is a need for low-speed control or positioning control (see Section 3.11).

Step 9: Tune positioning control parameters if necessary (see Section 3.12).

Step 10: Adjust field weakening control parameters as needed (see Section 3.13)

Step 11: By default, control commands are provided via UI software. To switch to external potentiometer control, adjust the control command source (see Section 3.14).

3.1 Modify motor control mode and parameters

- The *motor_control_drive_param.h* header file allows users to configure the motor drive mode, motor parameters, board parameters and controller parameters.
- Users can configure the desired drive mode according to their hardware and motor configurations, as detailed below.

1. Select to use an internal crystal.

Table 2. Macro definition setting (internal crystal)

Definition	Description
INTERNAL_CLOCK_SOURCE	Use an internal crystal (disabled by default)

2. Select a motor development board (V1 or V2, optional)

Table 3. Macro definition setting (EVB version)

Definition	Description
AT_MOTOR_EVB_V1	AT-MOTOR-EVB V1.x (include *mc_hwio_v1.h* file)
AT_MOTOR_EVB_V2	AT-MOTOR-EVB V2.0 (include *mc_hwio_v1.h* file)

3. Enable low-side MOS inverted output according to the model of gate driver on hardware. For example, U3315 on AT32_LV_MOTOR_CONTROL_EVB supports inverted output; therefore, the low-side MOS inverted output is enabled by default.

Table 4. Macro definition setting (low-side MOS inverted output)

Definition	Description
GATE_DRIVER_LOW_SIDE_INVERT	Enable low-side MOS inverted output (enabled by default; comment to disable)

4. This example is a FOC vector control demonstration project, so there is no need to make changes. This option, however, is reserved for use in code judgment.

Table 5. Macro definition setting (vector control mode)

Definition	Description
FOC_CONTROL	Vector control mode

5. Select a current sampling method from below according to user hardware circuit and application requirements

Table 6. Macro definition setting (current sampling method)

Definition	Description
THREE_SHUNT	3-shunt current sampling
TWO_SHUNT	2-shunt current sampling
ONE_SHUNT	1-shunt current sampling

6. If there is a need to use 2-shunt current sampling, select one from below according to user hardware circuit and application requirements; if not, this step is not needed.

Table 7. Macro definition setting (two-shunt current sampling)

Definition	Description
U_V_SHUNT	U & V shunt
V_W_SHUNT	V & W shunt
U_W_SHUNT	U & W shunt

7. Based on whether the user's hardware circuit uses the low-side MOSFET's RDS(on) for current sampling.

Table 8. Macro definition setting (MOSFET's $R_{DS(on)}$)

Definition	Description
MOS_RDS_SHUNT	Current Sampling Utilizing Low-Side MOSFET's $R_{DS(on)}$ (disabled by default)

8. Based on whether the user's requirements call for using dual ADCs for current sampling (Applicable to MCUs with dual or more ADCs; not suitable for ONE_SHUNT configurations).

Table 9. Macro definition setting (dual ADCs current sampling)

Definition	Description
TWO_ADC_CONVERTERS	Current Sampling with Dual-ADC Synchronization (disabled by default)

9. This example is for hall sensor project, so the hall sensor is selected by default, and there is no need to make changes. This option, however, is reserved for use in code judgment.

Table 10. Macro definition setting (hall sensor)

Definition	Description
HALL_SENSORS	Hall sensor

10. Enable low-speed voltage control according to actual needs (this is required for positioning control).

Table 11. Macro definition setting (low-speed voltage control)

Definition	Description
LOW_SPEED_VOLT_CTRL	Low-speed voltage control mode (disabled by default)

11. Enable D/Q-axis current signal low-pass filtering mode according to actual needs

Table 12. Macro definition setting (D/Q-axis current low-pass filter)

Definition	Description
CURRENT_LP_FILTER	D/Q-axis current low-pass filter signals (default OFF for 3 or 2-shunt current sampling, but default ON for 1-shunt current sampling)

12. Enable a/b/c phase current signal low-pass filter according to user needs.

Table 13. Macro definition setting (a/b/c phase current low-pass filter)

Definition	Description
AC_CURRENT_LP_FILTER	a/b/c phase current low-pass filter (disabled by default)

13. Enable field weakening feature during speed control according to actual needs.

Table 14. Macro definition setting (field weakening control)

Definition	Description
FIELD_WEAKENING	Field weakening control (default OFF)

14. Enable current decouple feature during torque (current) control according to user needs.

Table 15. Macro definition setting (current decouple control)

Definition	Description
CURRENT_DECOUPLE_CTRL	Current decouple control (N/A for AT32L021 series) (disabled by default)

15. Enable motor winding parameter identification feature. Once enabled, it is used to automatically detect RS and LS of the motor windings, and calculate current loop PI controller parameters (see Section 3.6).

Table 16. Macro definition setting (motor winding parameter identification)

Definition	Description
MOTOR_PARAM_IDENTIFY	Motor winding parameter identification (enabled by default; comment to disable)

16. Enable or disable the Artery motor UI tool according to user needs.

Table 17. Macro definition setting (Artery motor UI tool)

Definition	Description
USE_MOTOR_MONITOR	Enable Artery motor UI tool (enabled by default; comment to disable)

Table 18 gives the motor parameters used in the demo, such as number of pole-pairs, encoder parameters and Hall sensor parameters.

Table 18. Motor parameter definition

Definition	Description
RS_LL	Motor line-to-line winding resistance (unit: Ω)
LS_LL	Motor line-to-line winding inductance (unit: H)
POLE_PAIRS	Number of pole-pairs
KE	Motor KE value (unit: V/rpm) (fill in this value when current decouple control is enabled)
LD_LQ_RATIO	Ld to Lq ratio

Definition	Description
NOMINAL_CURRENT	Nominal current (unit: ampere)
HALL_LEARN_DIR	Motor direction after hall self-learning procedure
HALL_LEARN_0_STATE	Hall state at 0 degree after Hall self-learning session
HALL_LEARN_1_STATE	Hall state at 60 degree after Hall self-learning session
HALL_LEARN_2_STATE	Hall state at 120 degree after Hall self-learning session
HALL_LEARN_3_STATE	Hall state at 180 degree after Hall self-learning session
HALL_LEARN_4_STATE	Hall state at 240 degree after Hall self-learning session
HALL_LEARN_5_STATE	Hall state at 300 degree after Hall self-learning session

3.2 Connection settings

After the hardware and software are well prepared, set up connection between UI and the control board as follows. For details, refer to *AN0063_AT32 Motor Monitor Application Note*.

Step 1:

Connect the motor, AT-Link/J-Link and power supply to the motor control evaluation board, and connect USART interface to PC via an USB cable.

Step 2:

Use MDK to compile demo project code, and download code to the motor control evaluation board via J-Link or AT-Link.

Step 3:

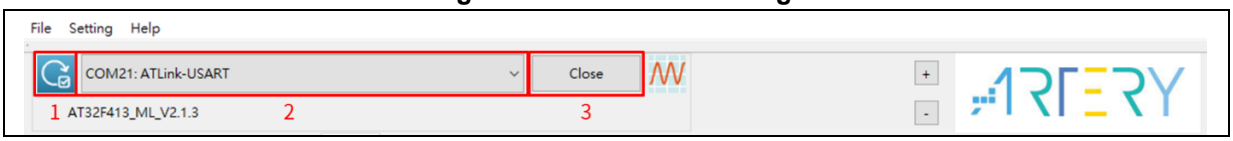
Run ArteryMotorMonitor.exe.

Go to File -> Open Project, and select ArteryMotorMonitor.atmcx-> Open.

Step 4:

Click the update icon (1.) of Serial Port and select the corresponding serial port (2.); then click Open(3.) to enable real-time communication, as shown in Figure 6.

Figure 6. Connection settings

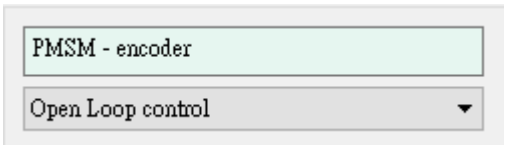


3.3 Open loop control

Using open-loop voltage control, this mode allows motor rotation without position sensors. It serves to confirm the motor's operational status and direction. The open-loop voltage and angular increment are adjusted incrementally from zero, based on the motor's speed and phase current.

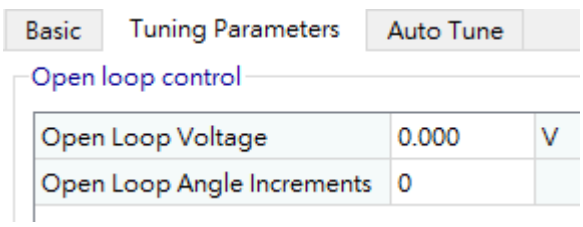
Step 1: Select “Open loop control” through the drop-down menu.

Figure 7. Select open loop control mode



Step 2: Go to “Tuning Parameters” to set “Open Loop Voltage” and “Open Loop Angle Increments”. Click on “Start Motor” and monitor the current until the motor starts to run properly (set the open loop voltage value appropriately to prevent overheating damage to the motor).

Figure 8. Open loop control parameters



Tips:

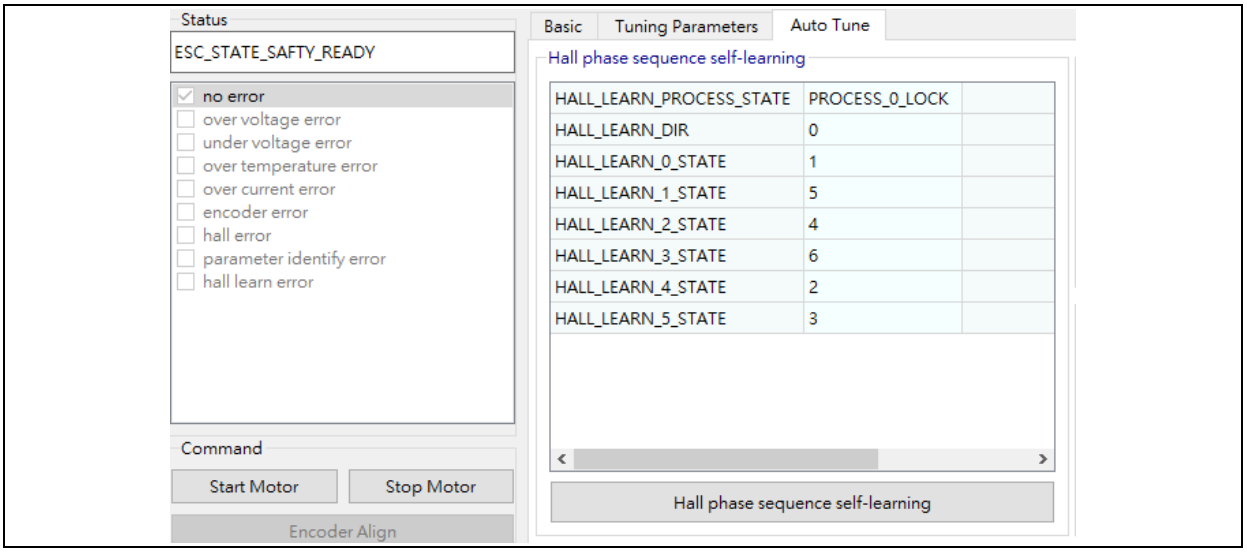
Initially, set “Open Loop Voltage” value to 0.5~2V and set “Open Loop Angle Increments” value to 1~10, and monitor whether the motor starts to run. If not, check whether the 3-phase cables are correctly connected.

3.4 Hall phase sequence self-learning

Step 1: Connect UI software.

Step 2: Enter debug mode and turn to Auto Tune page. When the status is ESC_STATE_SAFETY_READY, click on “Hall phase sequence self-learning”, as shown in Figure 9.

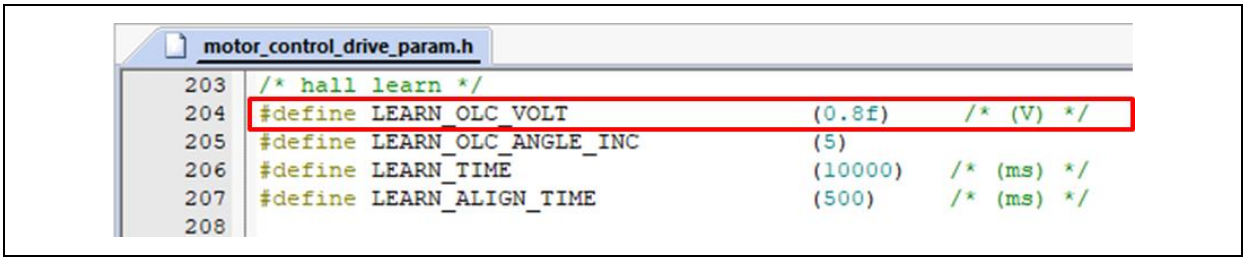
Figure 9. Hall phase sequence self-learning



Step 3: Motor starts running in open loop mode. If the motor rotating direction is found reverse, the user needs to wait until the motor stops before clicking on “Hall phase sequence self-learning” again to adjust its rotation direction. The Hall phase sequence self-learning feature is kept active until the motor stops working.

Note: If the motor failed to spin or work normally, it is necessary to modify the open loop voltage macro definition (LEARN_OLC_VOLT) and restart compiling and programming code (as shown in Figure 10), before restarting Step 2.

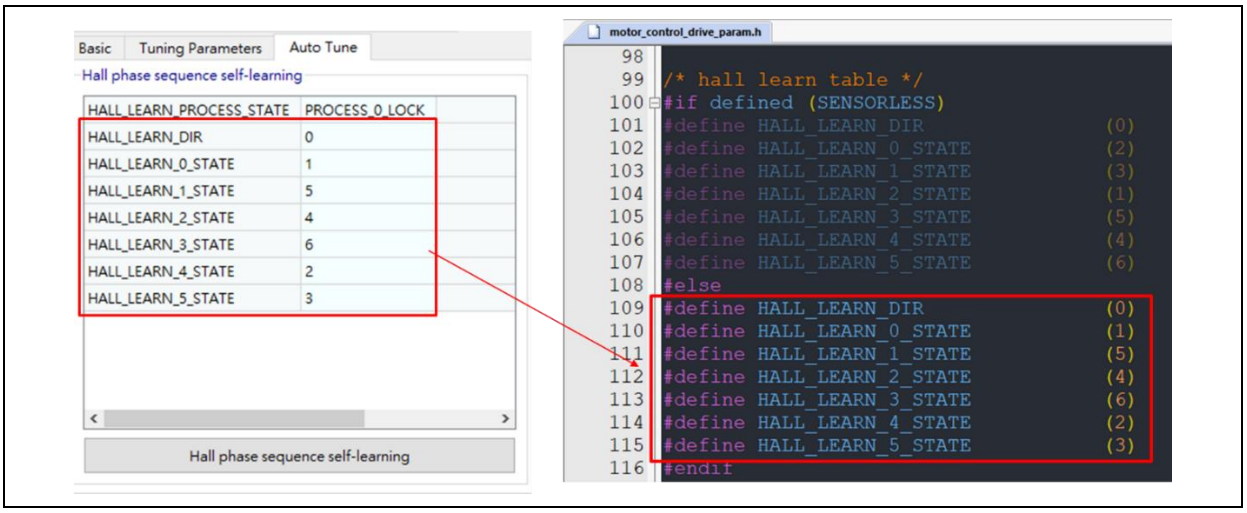
Figure 10. Hall phase sequence self-learning open loop voltage



Step 4: Confirm whether the HALL_LEARN_PROCESS_STATE is “PROCESS_4_FINISH” or not

Step 5: Write the values of the HALL_LEARN_DIR and HALL_LEARN_0_STATE~HALL_LEARN_5_STATE to the “hall learn table” macro definition (*motor_control_drive_param.h*) and restart compiling and programing code, as shown in Figure 11.

Figure 11. Hall phase sequence self-learning macro definition

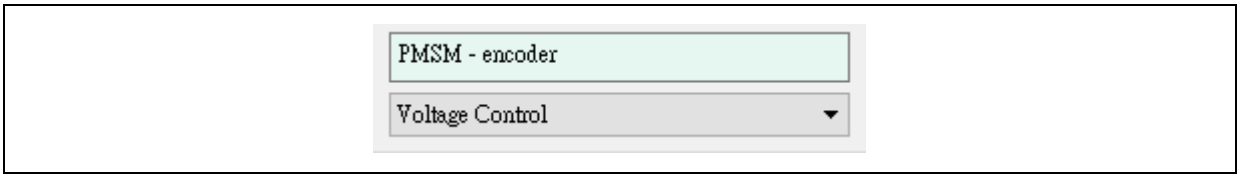


3.5 Voltage control

After the hall phase sequence self-learning is complete, it is possible to perform voltage control based on hall sensor position.

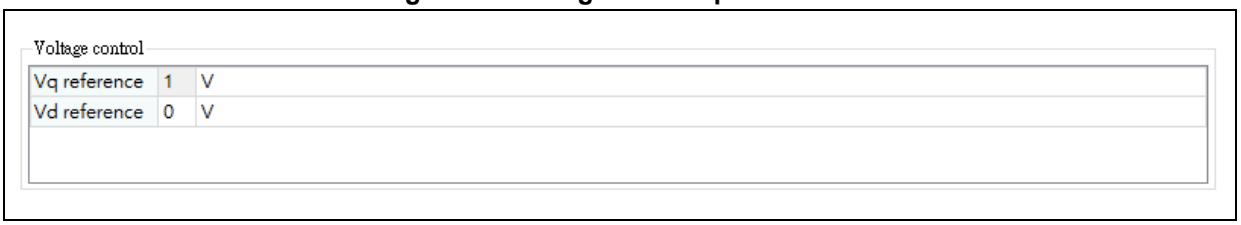
Step 1: Select “Voltage Control” through the drop-down menu.

Figure 12. Select Voltage control mode



Step 2: Use Q-axis voltage control to drive the motor, and view whether the Ia and Ib data match the actual currents, whether the current offset value is closer to 0 and whether the Ia phase leads Ib phase by 120 degrees (see Figure 33).

Figure 13. Voltage control parameters



3.6 Auto identification and tuning of parameters

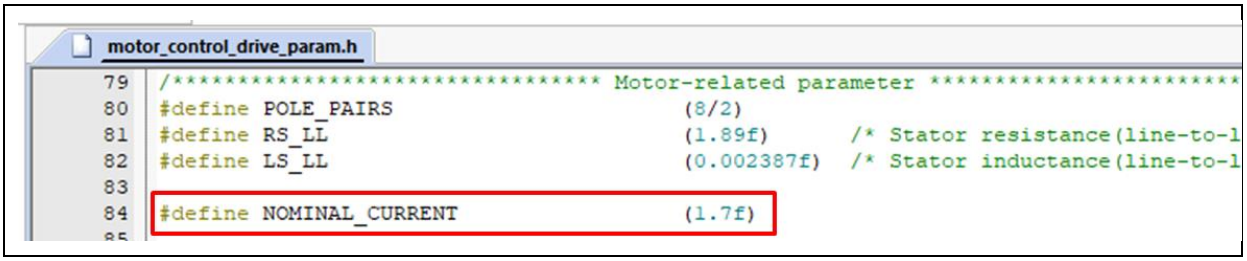
Automatic parameter identification is divided into two parts, namely automatic identification of motor winding parameters and self-tuning of current PI controller parameters.

Auto identification of motor winding parameters are not required if there are RS_LL and LS_LL parameters. Users can directly modify macro definitions of RS_LL and LS_LL in *motor_control_drive_param.h* file.

Auto identification of motor winding parameters:

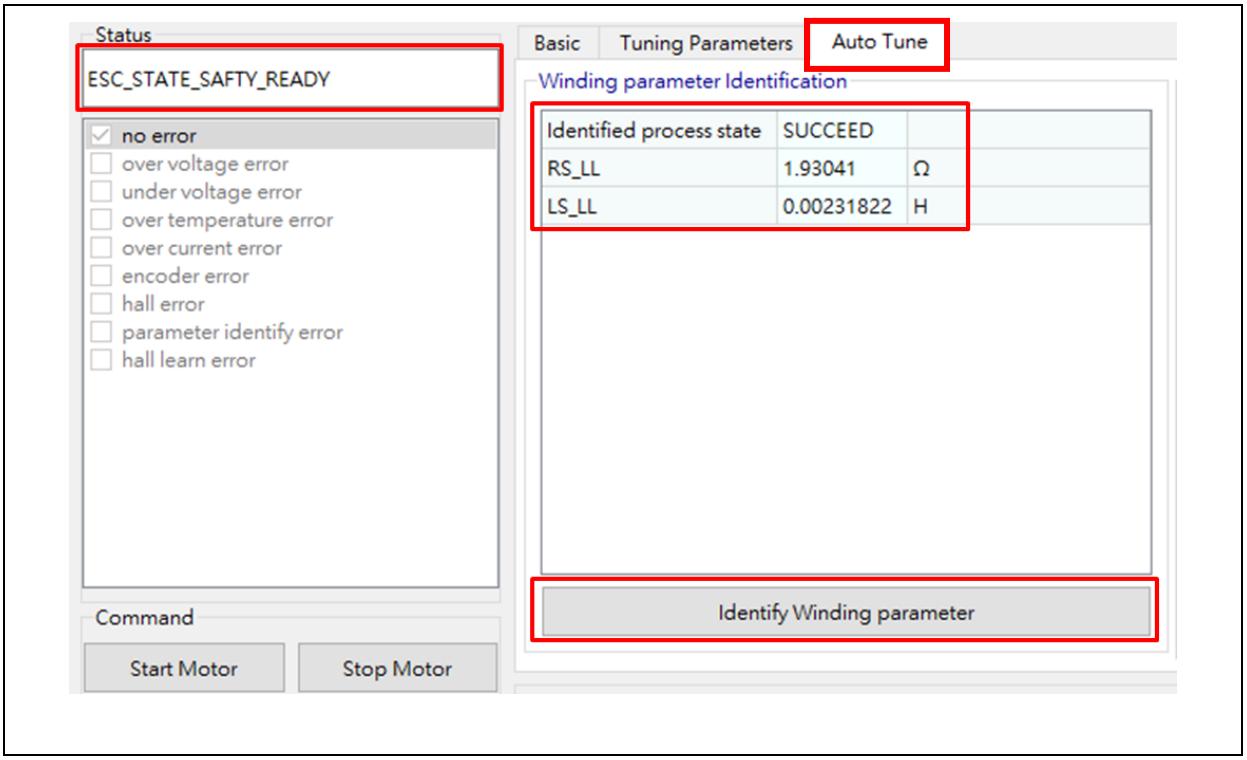
Step 1: Write the macro definition NOMINAL_CURRENT into *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 14.

Figure 14. Nominal current macro definition



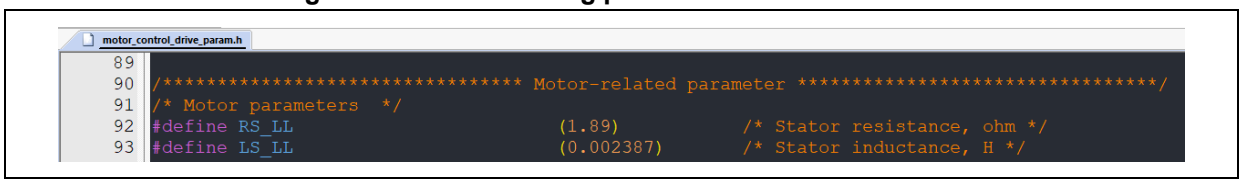
Step 2: Go to UI --> Auto Tune, and click on “Identify Winding parameter” in “safety ready” status to perform winding parameter identification, as shown in Figure 15.

Figure 15. Identify winding parameters



Step 3: Check and confirm “SUCCEED” of “Identified process state”; get the RS_LL and LS_LL values and fill them into the corresponding macro definitions in *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 16.

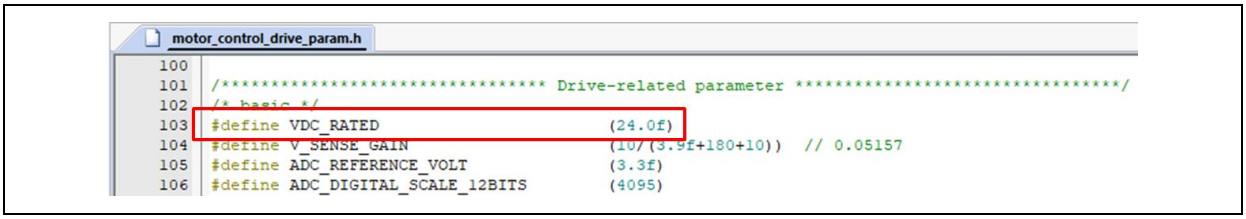
Figure 16. Motor winding parameters macro definition



Current PI controller parameter auto tuning:

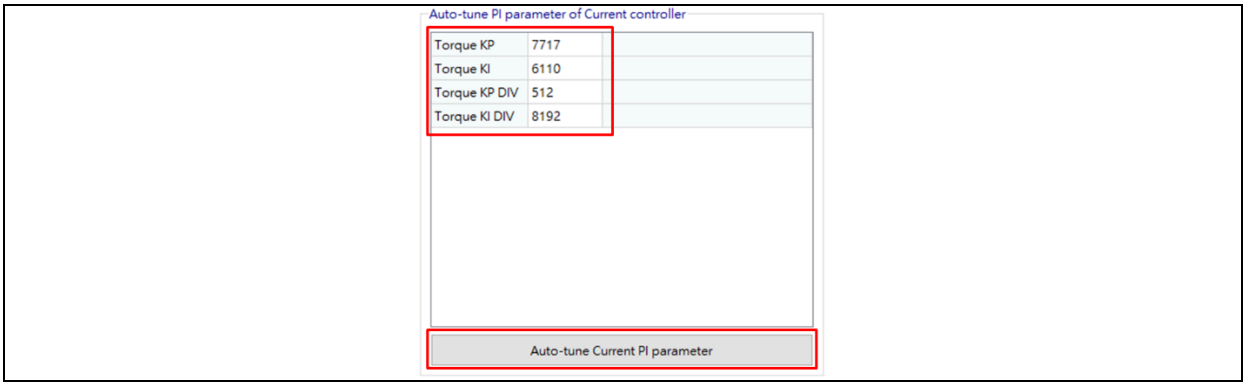
Step 1: Fill in the macro definition of input DC voltage (VDC_RATED) into the *motor_control_drive_param.h* file, and then re-compile and re-program code, as shown in Figure 17.

Figure 17. Nominal voltage



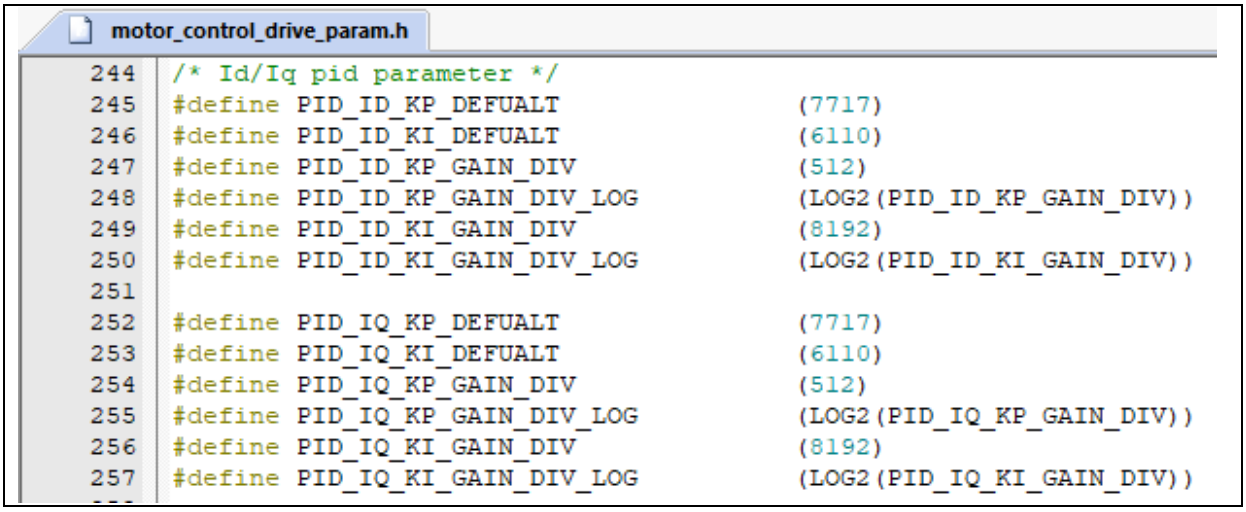
Step 2: Go to Auto Tune window, and click on “Auto-tune Current PI parameter” in “safety ready” status to perform auto tuning of current PI controller parameters. After auto tuning is completed, current Q axis and D axis PI controller parameters are automatically updated, as shown in Figure 18.

Figure 18. Auto tune current PI parameters



Step 3: Fill the parameters shown in Figure 18 into the corresponding macro definition (motor_control_drive_param.h), then recompile and re-program code, as shown in Figure 19.

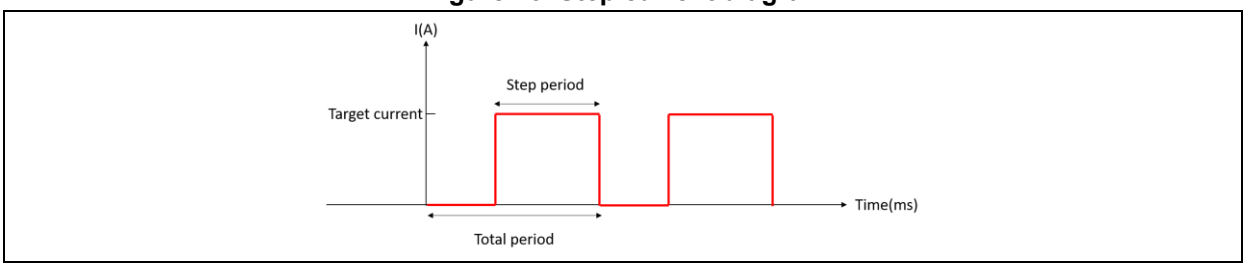
Figure 19. Macro definition of current PI parameters



3.7 ID tune

A step current is generated in IQ Tune mode, as shown in Figure 20. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of D axis current.

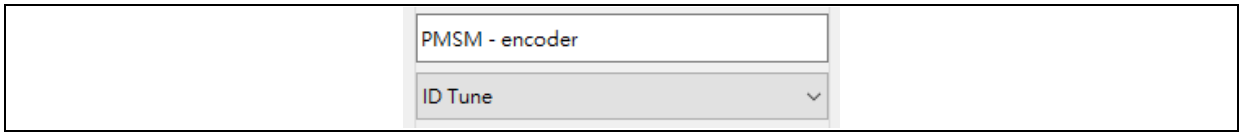
Figure 20. Step current diagram



Note: It is recommended to tune D-axis current and then Q-axis current, and to set KP and KI to 0 for D-axis and Q-axis current to avoid unmatched default values affecting current parameter adjustment.
Note: It is recommended to set D-axis current command to 5-10% of the rated current and then slowly increase the D-axis current to the target value to align the rotor with D-axis, so that to protect IPM or MOSFET components from being burnt out by current surges.

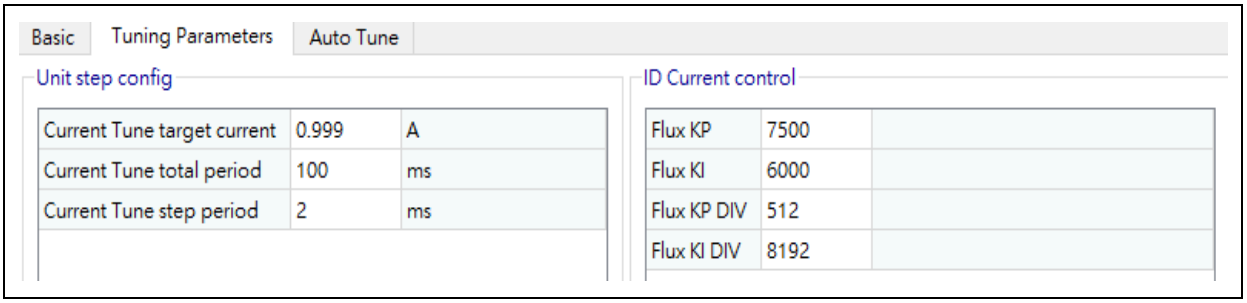
Follow the steps bellow:
Step 1: Select “ID tune” through the drop-down menu.

Figure 21. Select ID Tune mode



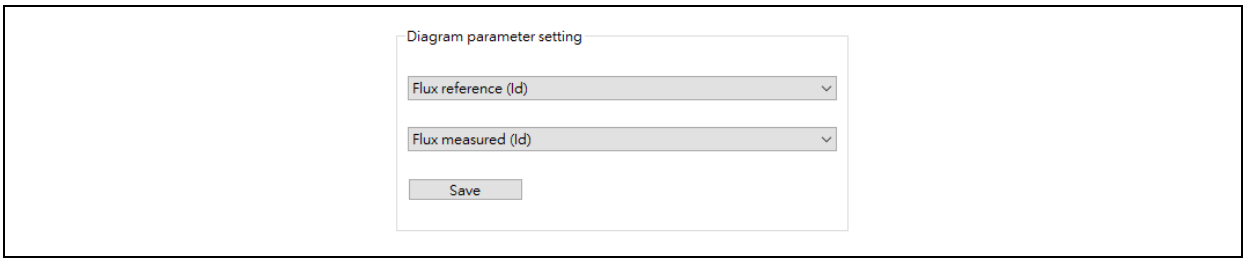
Step 2: Go to “Tuning Parameters” window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 22.

Figure 22. D-axis current PID and step current parameters



Step 3: Click on “Start Motor”.
Step 4: Set “Flux reference(Id)” and “Flux measured (Id)” in “Diagram parameter setting”, and then click on “Save”.

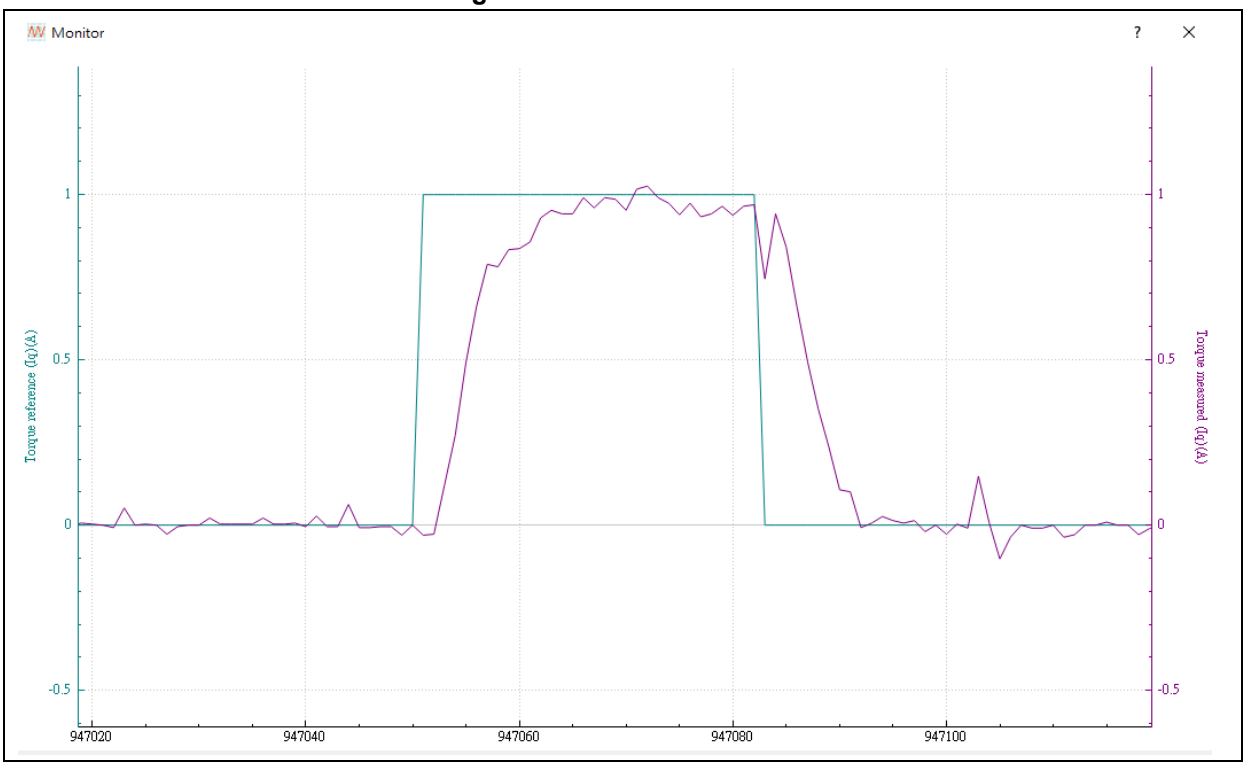
Figure 23. Diagram parameter settings (ID tune)



Step 5: Click icon to generate waveforms.

Step 6: Check whether the current response is as expected, as shown in Figure 24. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 24. ID tune waveform



STEP-7: After ID tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 25.

Figure 25. D-axis current PI control parameter macro definition

motor_control_drive_param.h

```
245 #define PID_ID_KP_DEFUALT                (7500)
246 #define PID_ID_KI_DEFUALT                (6000)
247 #define PID_ID_KP_GAIN_DIV               (512)
248 #define PID_ID_KP_GAIN_DIV_LOG           (LOG2(PID_ID_KP_GAIN_DIV))
249 #define PID_ID_KI_GAIN_DIV               (8192)
250 #define PID_ID_KI_GAIN_DIV_LOG           (LOG2(PID_ID_KI_GAIN_DIV))
```

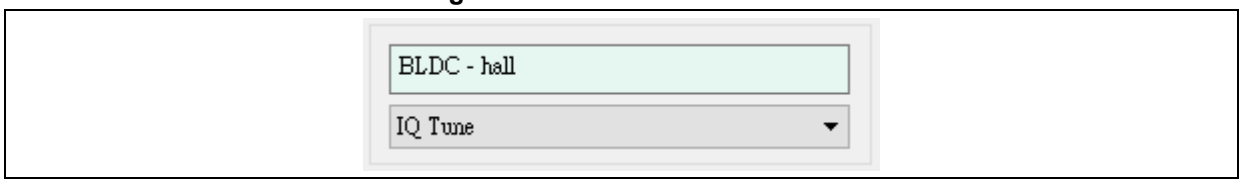
3.8 IQ tune

A step current is generated in IQ Tune mode, as shown in Figure 20. Parameters related to the step current are adjustable. The step current is generated to help check the current response after adjusting PID parameters of Q axis current.

Follow the steps bellow:

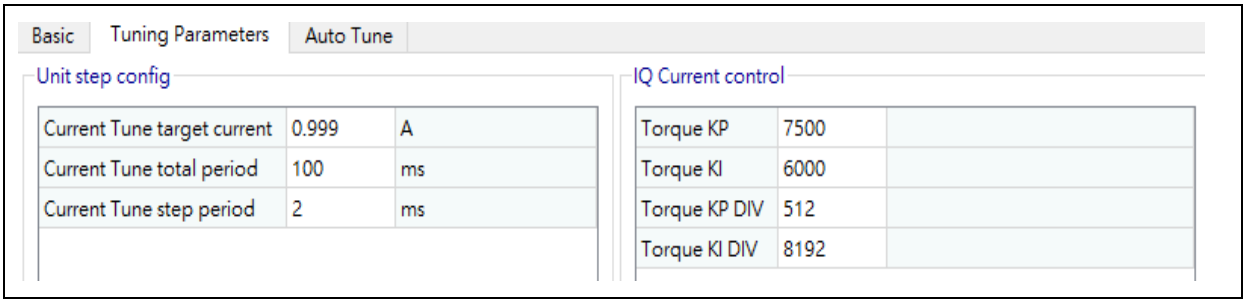
Step 1: Select “IQ tune” through the drop-down menu.

Figure 26. Select ID Tune mode



Step 2: Go to “Tuning Parameters” window to set PID parameters and step current parameters. Parameters obtained by current PI controller parameters auto tuning can be used for initial adjustment, and users can adjust these parameters as needed to get the desired current response, as shown in Figure 27.

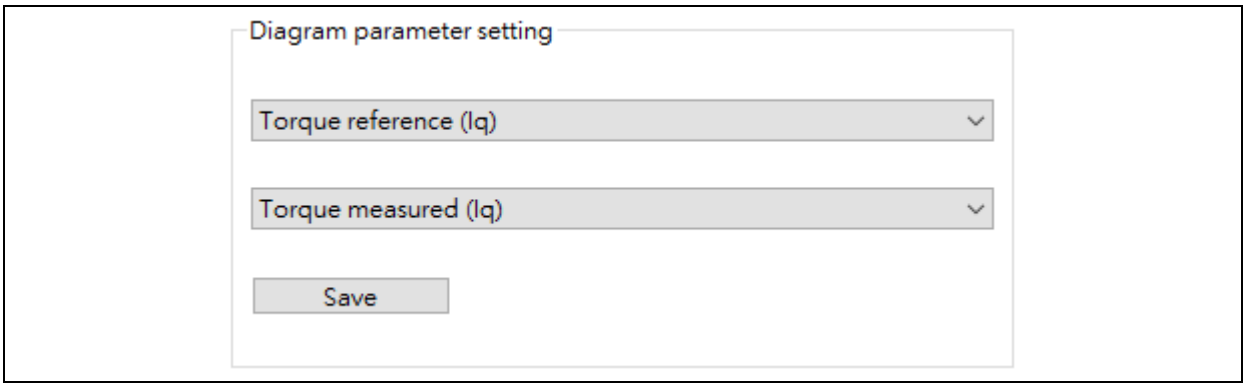
Figure 27. Q-axis current PID and step current parameters



Step 3: Click on “Start Motor”.

Step 4: Set “Torque reference (Iq)” and “Torque measured (Iq)” in “Diagram parameter setting”, and then click on “Save”.

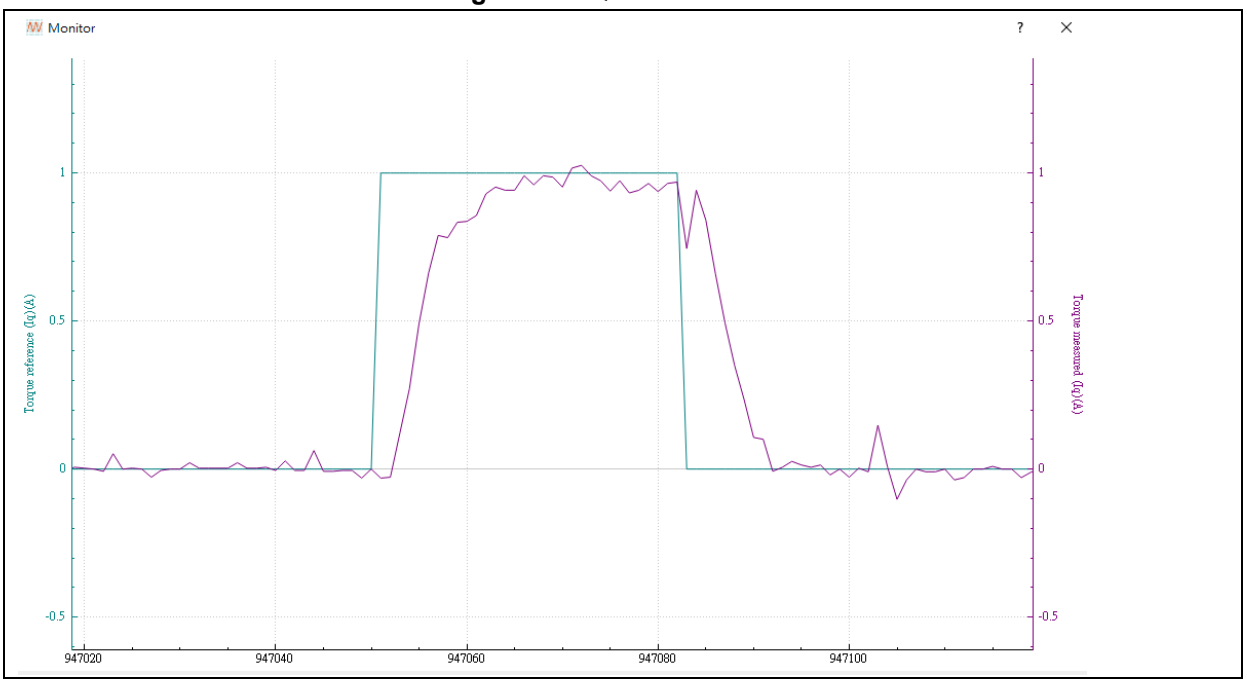
Figure 28. Diagram parameter settings (IQ tune)



Step 5: Click  icon to generate waveforms. For more information, please refer to *AN0063 AT32 Motor Monitor Application Note*

Step 6: Check whether the current response is as expected, as shown in Figure 29. If not, click to stop the motor and repeat STEP-2~STEP-6.

Figure 29. IQ tune waveform



STEP-7: After IQ tuning is completed, fill parameters into the macro definition (motor_control_drive_param.h), and then re-compile and re-program code, as shown in Figure 30.

Figure 30. Q-axis current PI control parameter macro definition

motor_control_drive_param.h

```
252  #define PID_IQ_KP_DEFUALT              (7500)
253  #define PID_IQ_KI_DEFUALT              (6000)
254  #define PID_IQ_KP_GAIN_DIV             (512)
255  #define PID_IQ_KP_GAIN_DIV_LOG         (LOG2(PID_IQ_KP_GAIN_DIV))
256  #define PID_IQ_KI_GAIN_DIV             (8192)
257  #define PID_IQ_KI_GAIN_DIV_LOG         (LOG2(PID_IQ_KI_GAIN_DIV))
```

3.9 Current loop control

In this mode, the current loop control is used to control torque by tuning the target current. The results are reflected through waveforms.

Follow the steps below:

Step 1: Select “Torque Control” through the drop-down menu.

Step 2: Set software control as control source and configure the target current reference (Torque reference).

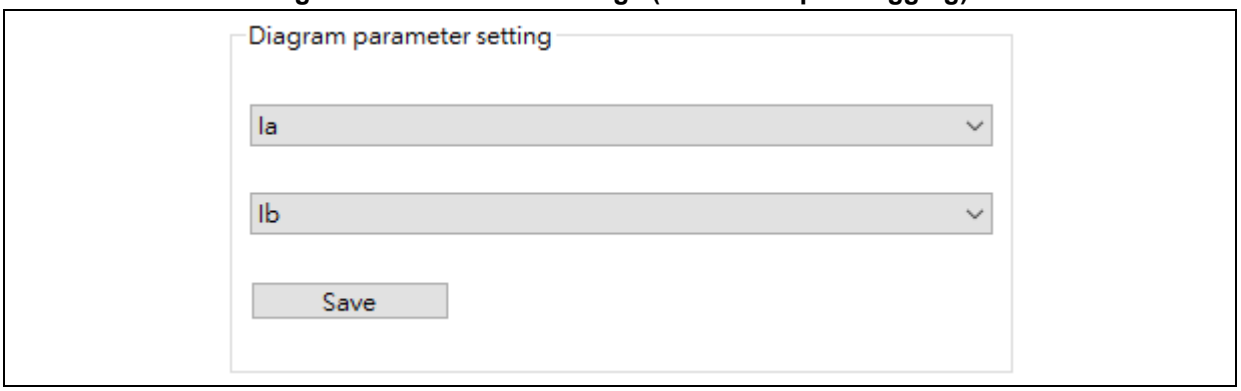
Figure 31. Set target current

Torque reference (Iq)	0.100	A
Flux reference (Id)	0.000	A

Step 3: Click on “Start Motor”

Step 4: Set “Ia” and “Ib”, and then click on “Save”.

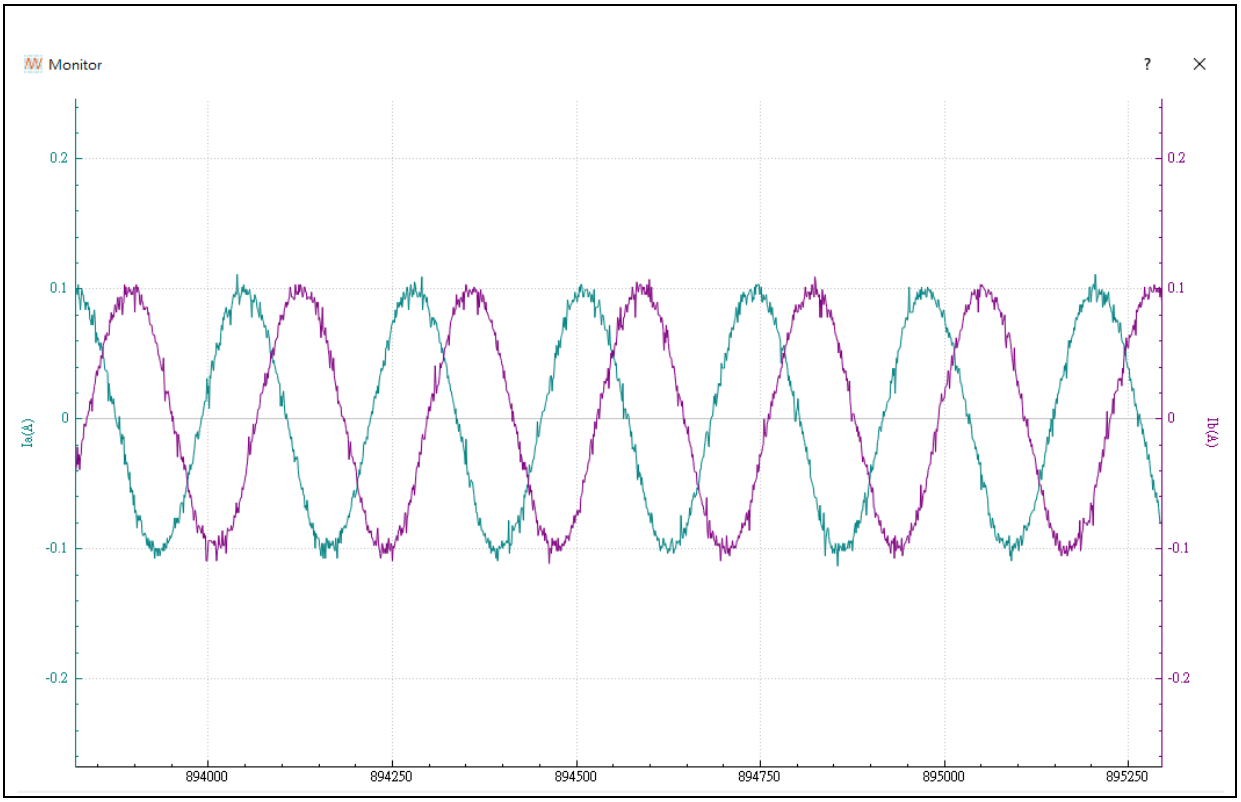
Figure 32. Parameter settings (current loop debugging)



Step 5: Click on “ ” to open a waveform window.

Step 6: Check if the current response is as expected, as shown in Figure 33.

Figure 33. Current loop debug waveforms



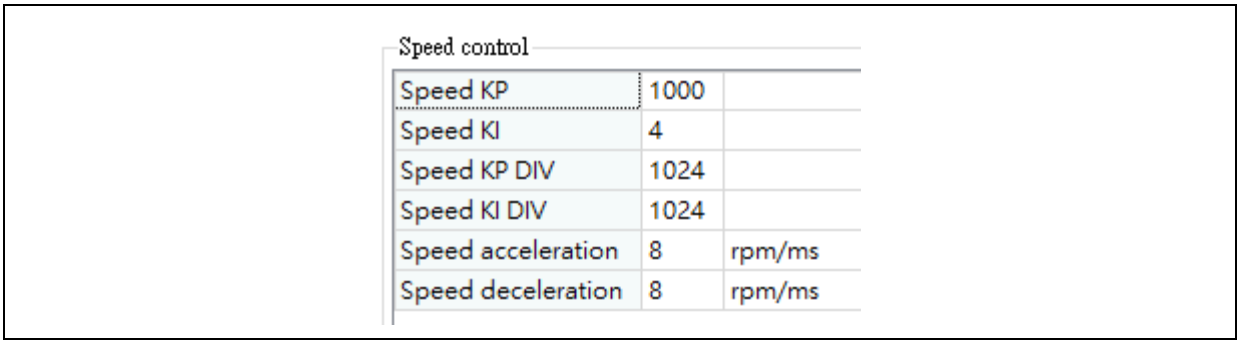
3.10 Speed loop control

In Speed control mode, the user can set speed PID parameters, acceleration and deceleration, and check the response through waveform. Follow the steps bellow:

Step 1: Select “Speed Control” through the drop-down menu.

Step 2: Go to “Tuning Parameters” window, and set speed PID parameters, acceleration and deceleration.

Figure 34. PID parameters, acceleration and deceleration



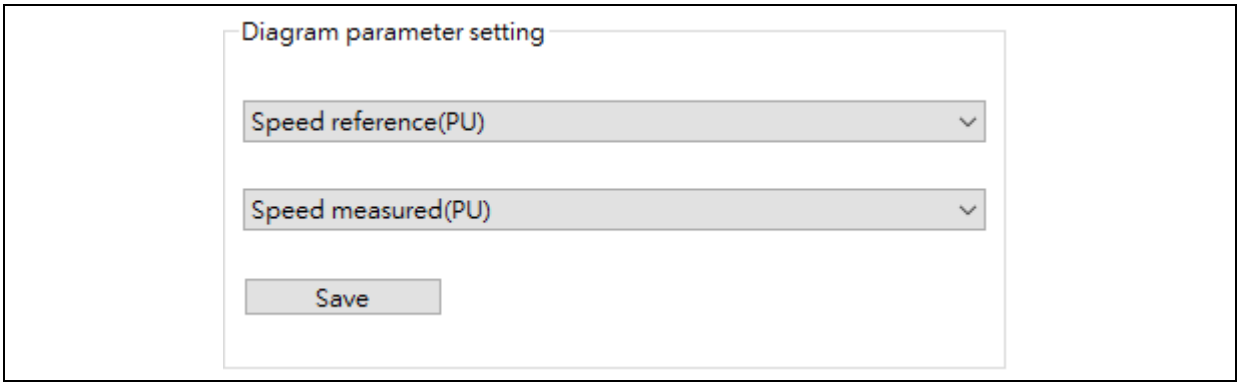
Step 3: Select “software control” in “Control source” and set the target speed (Speed reference).

Figure 35. Set target speed

Step 4. Click on “Start Motor”.

Step 5. Set “Speed reference” and “Speed measured” in “Diagram parameter setting”, and then click on “Save”.

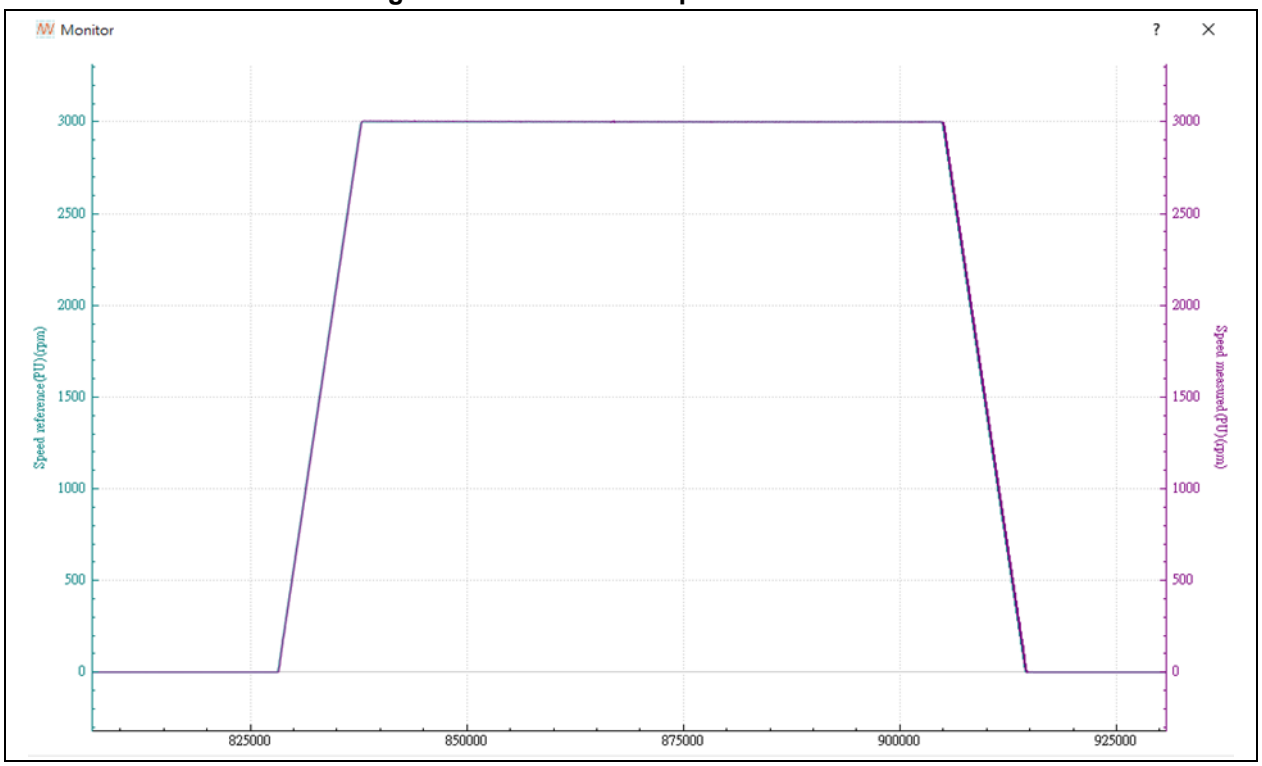
Figure 36. Diagram parameter settings (speed control)



Step 6. Click  icon to generate waveforms.

Step 7. Check if the speed response is as expected, as shown in Figure 37. If not, click to stop the motor and repeat STEP-2~STEP-7.

Figure 37. Waveform in speed control mode



Note: The maximum displayed values for speed waveforms (Speed Reference (PU) and Speed Measured (PU)) are defined as 1.25 × MAX_SPEED_RPM, with the minimum being -1.25 × MAX_SPEED_RPM. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to set MAX_SPEED_RPM to the motor's rated speed to avoid display anomalies.

3.11 Low-speed voltage control mode

The low-speed voltage control mode is required for low-speed control or positioning control applications. In this mode, the user can set PID parameters of low-speed voltage loop control, the minimum speed value for switching to voltage control, and the maximum speed value for switching to current control mode. The results are shown in waveforms generated. Follow the steps below:

Step 1: Define the LOW_SPEED_VOLT_CTRL in the *motor_control_drive_param.h*.

Figure 38. Low speed voltage control definition

```
59 #ifdef HALL_SENSORS
60 #define LOW_SPEED_VOLT_CTRL      /* if need low speed control or position control */
61 #endif
```

Step 2: Set the minimum speed value for switching from current control to voltage control (HYSTERESIS_LOW_SPEED) and the maximum speed value for switching from voltage control to current control (HYSTERESIS_HIGH_SPEED).

Step 3: Set PID_SPD_VOLT_KP_DEFUALT, PID_SPD_VOLT_KI_DEFUALT and PID_SPD_VOLT_KD_DEFUALT to 0, and then re-compile and re-program code.

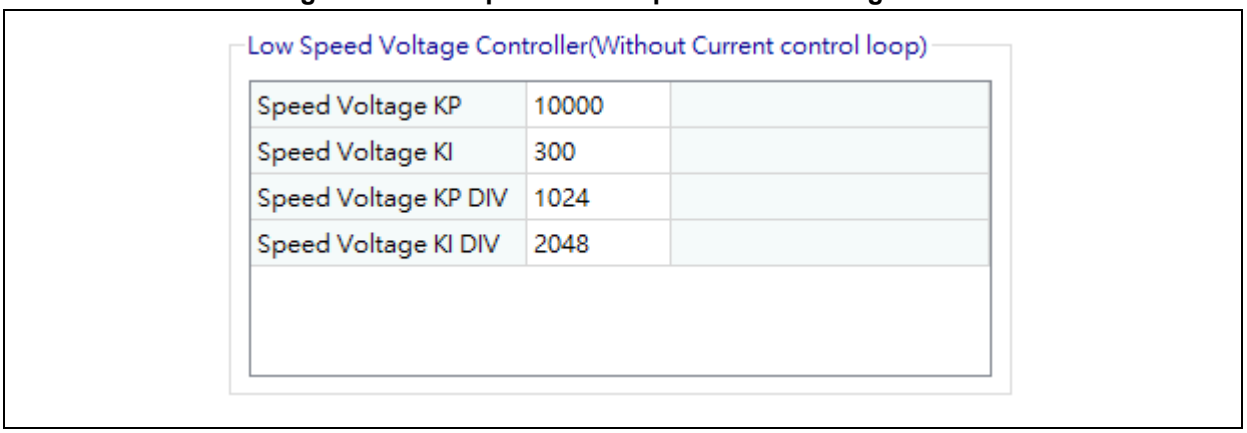
Figure 39. Low speed voltage control parameter definition

```
/* low speed voltage control parameter */
#define HYSTERESIS_LOW_SPEED            300
#define HYSTERESIS_HIGH_SPEED           400
/* low speed pid parameter for voltage control */
#define PID_SPD_VOLT_KP_DEFUALT         0
#define PID_SPD_VOLT_KI_DEFUALT         0
#define PID_SPD_VOLT_KD_DEFUALT         0
#define PID_SPD_VOLT_KP_GAIN_DIV        1024
#define PID_SPD_VOLT_KP_GAIN_DIV_LOG    LOG2(PID_SPD_VOLT_KP_GAIN_DIV)
#define PID_SPD_VOLT_KI_GAIN_DIV        2048
#define PID_SPD_VOLT_KI_GAIN_DIV_LOG    LOG2(PID_SPD_VOLT_KI_GAIN_DIV)
```

Step 4. Select “Speed Control” mode through the drop-down menu.

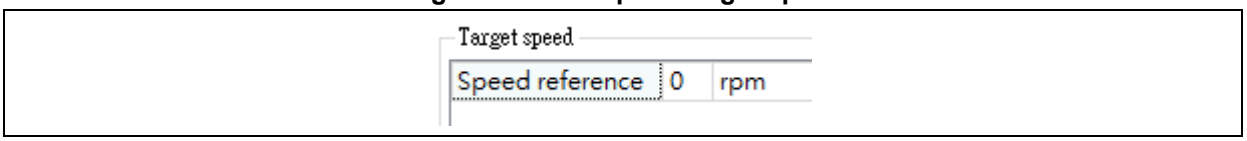
Step 5. Navigate to the Tuning Parameters page and configure PID parameters for Low-Speed Voltage Controller.

Figure 40. Low speed control parameters configuration



Step 6. Select “software control” in “Control source”, and set Speed reference, such as 300 rpm.

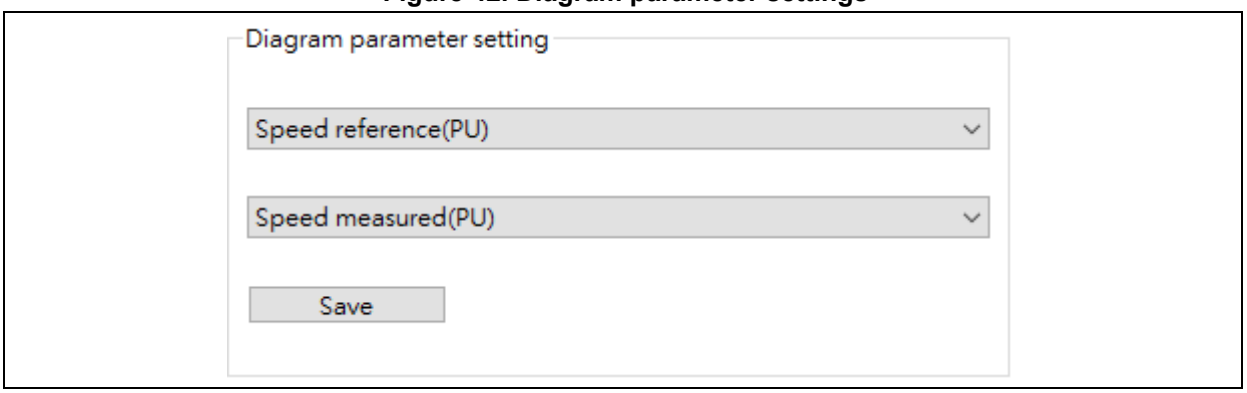
Figure 41. Low speed target speed



Step 7. Click on “Start Motor”.

Step 8. Go to “Diagram parameter settings” to set “Speed reference (PU)” and “Speed measured (PU)”, and then click on “Save”.

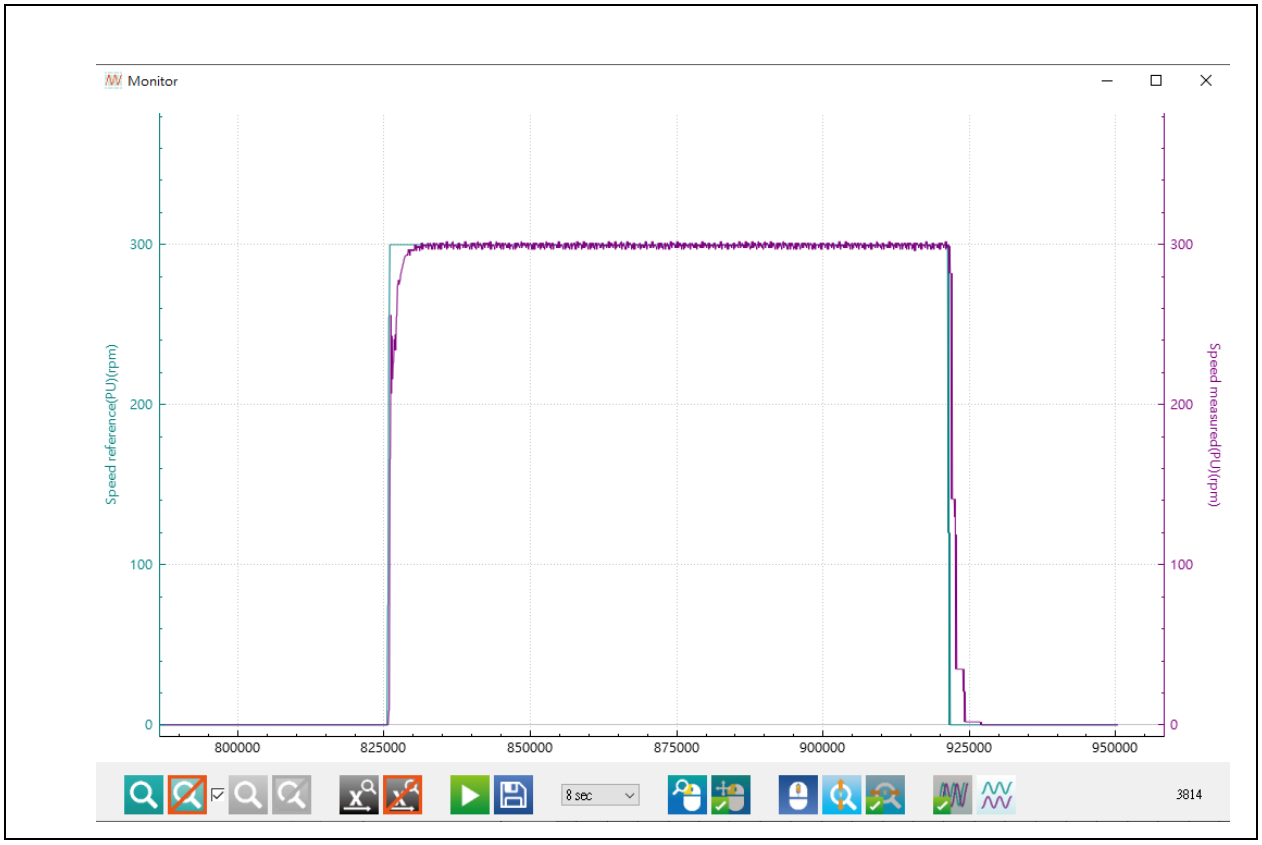
Figure 42. Diagram parameter settings



Step 9. Click  icon to generate waveforms.

Step 10. Check whether the speed response is as expected (Figure 43). If not, click to stop the motor and repeat Step 5~Step 10.

Figure 43. Low speed waveforms



3.12 Position loop control

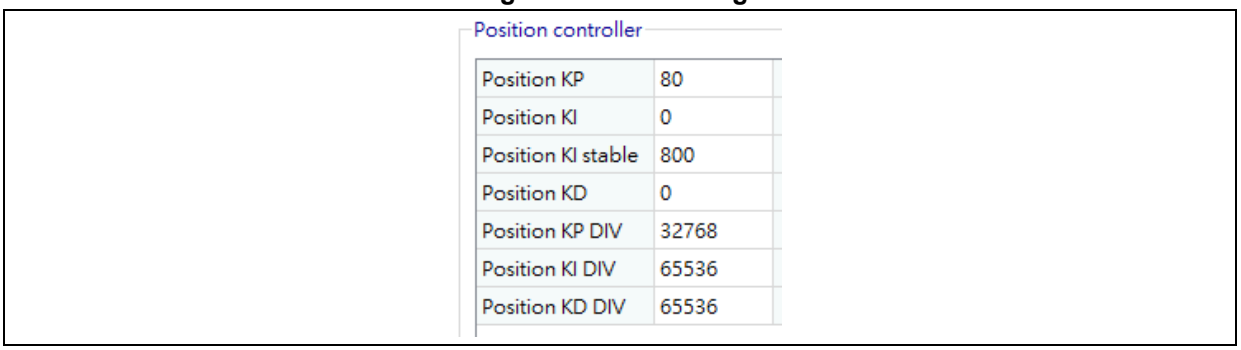
Hall sensor-based position detection mode has only a 60 electrical angle in resolution, which makes it not as accurate as that of encoder. The accuracy for positioning control should be ±120 electrical angle, therefore it is recommended to use motor in conjunction with a speed reducer to acquire better mechanical angle control. In this mode, the user can adjust positioning loop PID parameters, and view the results through waveforms generated. Follow the steps below:

Step 1: Select “Position Control” through the drop-down menu. If the motor has rotated, its angle usually will not stop exactly at the Zero degrees, so in this case, the positioning command reference and the measured position are the angle after the motors rotation.

Figure 44. Position reference and measured position

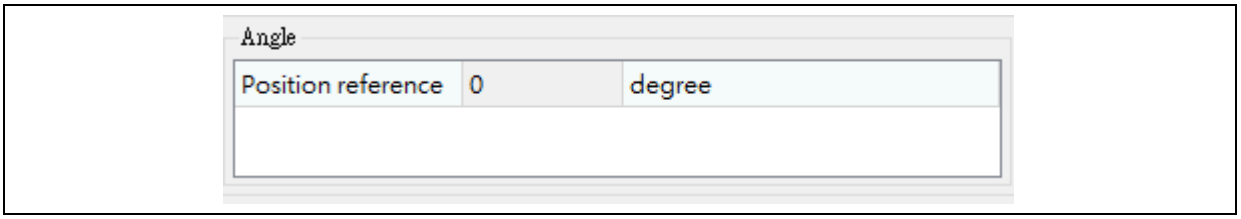
Step 2: Set position controller PID parameters.

Figure 45. PID settings



Step 3: Set Position reference, such as 36000 degrees.

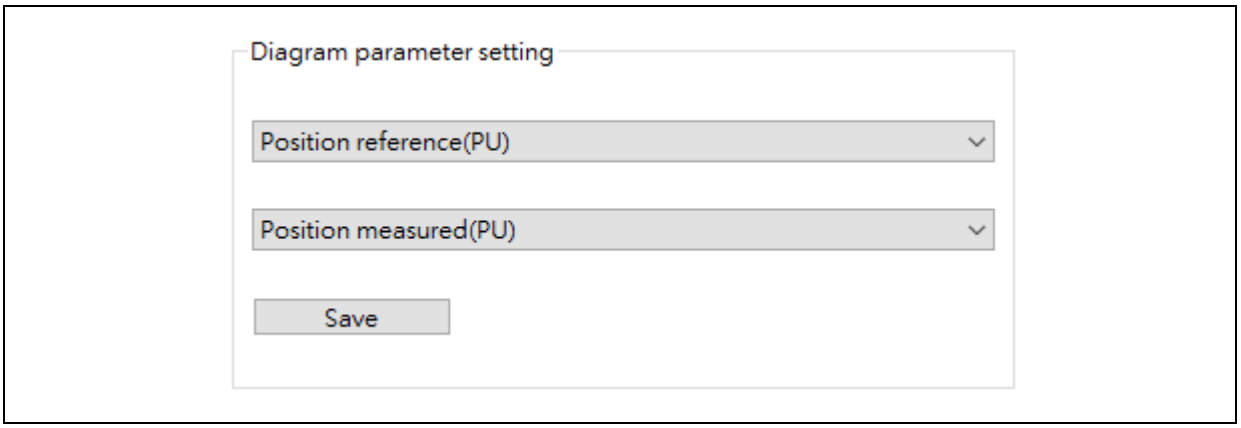
Figure 46. Set position reference



Step 4: Click on “Start Motor”.

Step 5: Go to “Diagram parameter setting” to tune “Position reference(PU)” and “Position measured(PU)”, and then click on “Save”.

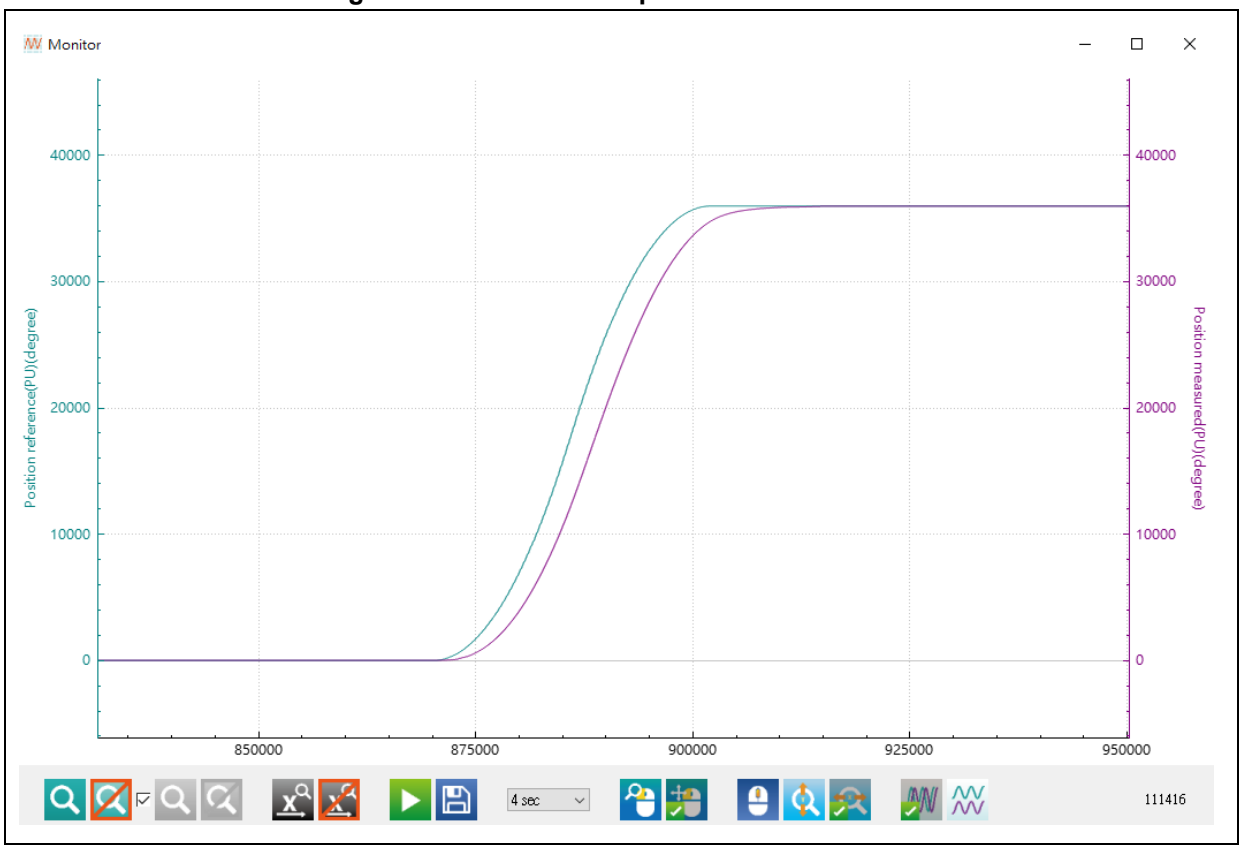
Figure 47. Diagram parameter setting (position loop control)



Step 6. Click  icon to generate waveforms.

Step 7. Check whether the position response is as expected, as shown in Figure 48. If not, click to stop the motor and repeat Step 2~Step 7.

Figure 48. Waveforms in position control mode



Note: The maximum displayed values for position waveforms (Position reference (PU) and Position measured (PU)) are defined as 1.25 × MAX_POSITION_ANGLE, with the minimum being -1.25 × MAX_POSITION_ANGLE. If values exceed these limits, the waveform display will clip (overflow), though the actual numerical values remain unaffected. It is recommended to adjust MAX_POSITION_ANGLE value appropriately to avoid display anomalies.

3.13 Flux-Weakening Control (Surface-Mounted Permanent Magnet Synchronous Motor, SPMSM)

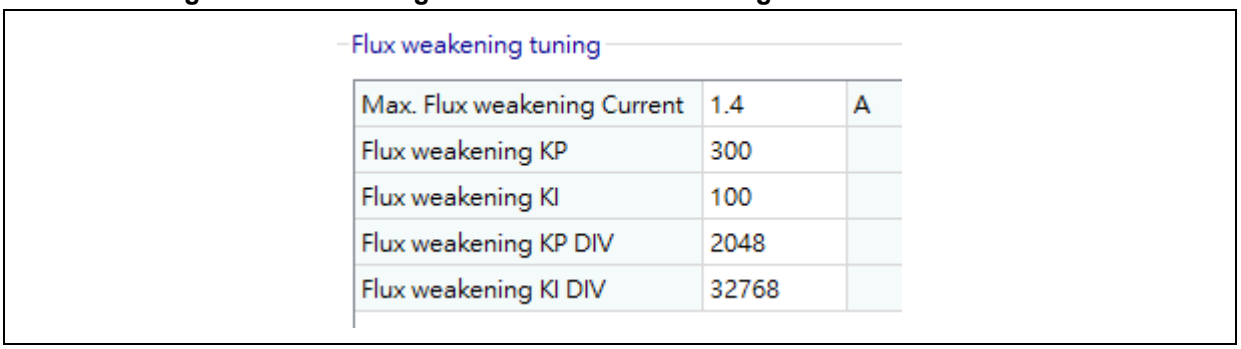
This mode supports Surface-Mounted Permanent Magnet Synchronous Motors (SPMSM). During speed-loop control, it injects negative d-axis current commands to weaken the permanent magnet flux linkage, thereby extending the motor's operational speed range. Adjustable parameters include the flux-weakening control (FWC) PID gains and the maximum negative d-axis current limit. After tuning, the dynamic response can be visualized via waveform plotting. Detailed implementation steps are as follows:

Step 1: Select "Speed Control" through the drop-down menu.

Step 2. Navigate to the Tuning Parameters page, configure the maximum d-axis (flux-weakening) current command and adjust the flux-weakening PID parameters.

Note: Configure the maximum d-axis (flux-weakening) current command according to the motor's rated specifications. Exceeding this limit may cause irreversible demagnetization of permanent magnets or thermal damage due to excessive copper/core losses.

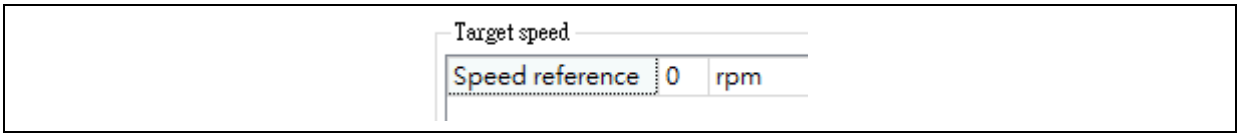
Figure 49. FWC PID gains and the maximum negative d-axis current limit



Step 3. Set Control Source to Software Control and input the Speed Reference value. For example: 8000rpm.

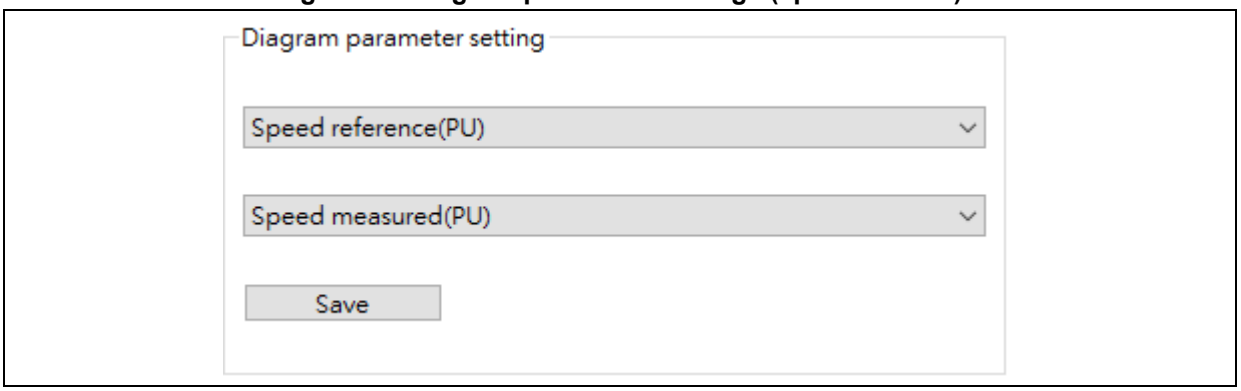
Note: The flux-weakening control (FWC) is triggered only when the motor speed exceeds the base speed (i.e., the maximum sustainable speed under rated flux linkage).

Figure 50. Target speed setting (FWC)



Step 4: Select Speed reference(PU) and Speed measured(PU), and click on "Save".

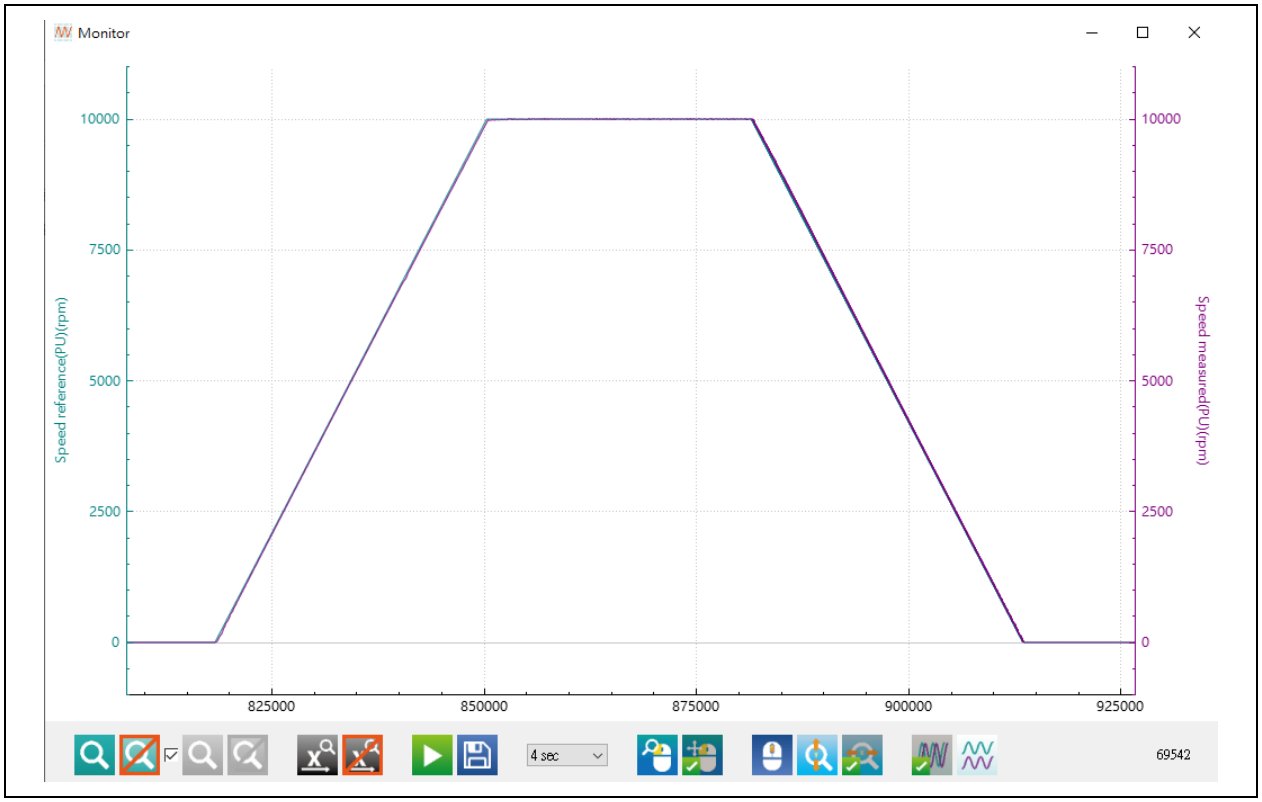
Figure 51. Diagram parameter settings (speed control)



Step 5: Click  icon to generate waveforms.

Step 6: Click on “Start motor” to check whether the speed response is as expected, as shown in Figure 52. If not, click to stop the motor and repeat Step 2~Step 6.

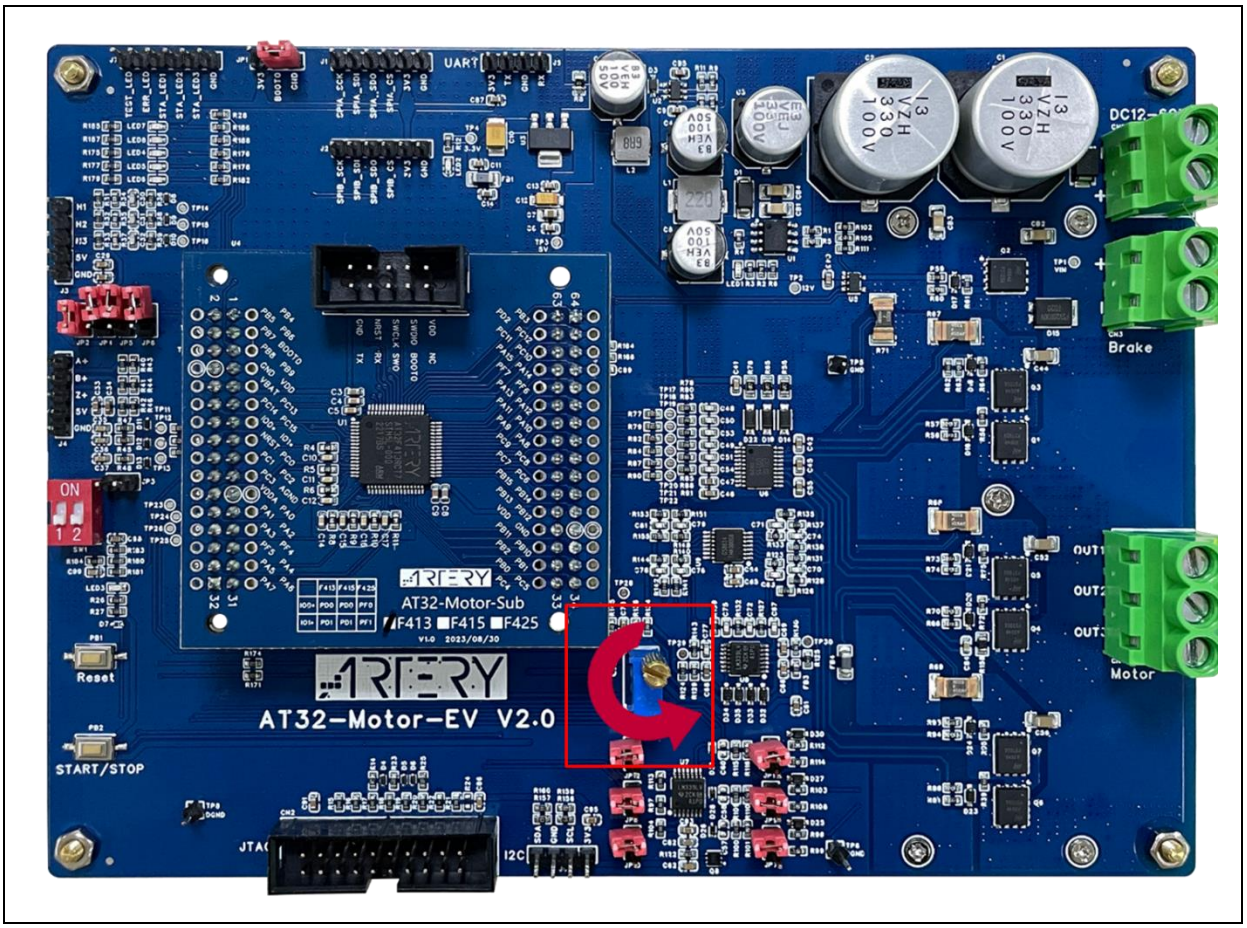
Figure 52. Waveforms in speed control mode (FWC)



3.14 External voltage control/software control mode

This demo project support two control sources, i.e., external voltage control and software control. The external voltage control mode adjusts speed or torque through external voltage. Users can adjust the potentiometer (as shown in the red box below) to control the command value. The voltage increases when the potentiometer is turned **counterclockwise**. The software control mode inputs speed or torque command value for control purpose. In this demo project, software control mode is set by default.

Figure 53. External voltage control – potentiometer



Users can select the control source by modifying the corresponding definition (CTRL_SOURCE in *motor_control_drive_param.h* file, as shown in Figure 54) or through PC software (see Figure 55). Control parameters are different for different control modes, such as the target speed (as shown in Figure 55) in speed control mode, or torque reference (as shown in Figure 56) in torque control mode.

Figure 54. Control source definition (software control (upper)/external voltage control (lower))

motor_control_drive_param.h

```
183
184 /********************************* Control-related parameter ************************
185 #define PWM_FREQ                    (30000)                          /*!< Hz */
186 #define MOTOR_CONTROL_MODE          (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE                 (ctrl_source_type)(CTRL_SOURCE_SOFTWARE)
188 #define UI_UART_BAUDRATE            (1500000UL)
189 #define UI_SAMPLE_CYCLE             (1)
```

```
motor_control_drive_param.h
183
184 /******************************** Control-related parameter ************************
185 #define PWM_FREQ                    (30000)                          /*!< Hz */
186 #define MOTOR_CONTROL_MODE          (motor_control_mode)(SPEED_CTRL)
187 #define CTRL_SOURCE                 (ctrl_source_type)(CTRL_SOURCE_EXTERNAL)
188 #define UI_UART_BAUDRATE            (1500000UL)
189 #define UI_SAMPLE_CYCLE             (1)
```

1) Select “External voltage control” as a control source

Select “external voltage control” in “Control source”. In this mode, it is possible to adjust speed or torque value by setting external voltage values. The “Target speed” and “Torque reference” area will generate the control speed or torque calculated based on the current voltage.

Note: The “Target speed” and “Torque reference” cannot be modified in “external voltage control” mode.

Figure 55. Speed control mode (external voltage control)

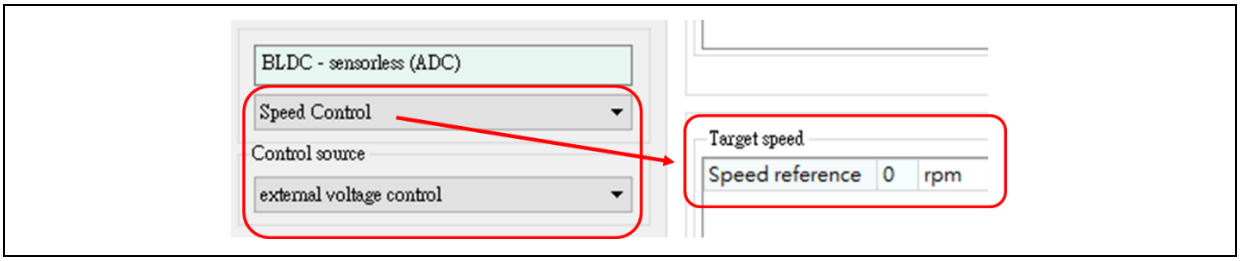
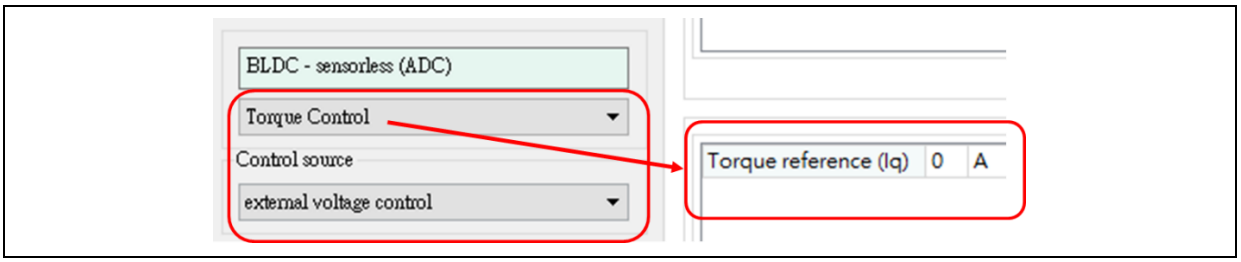


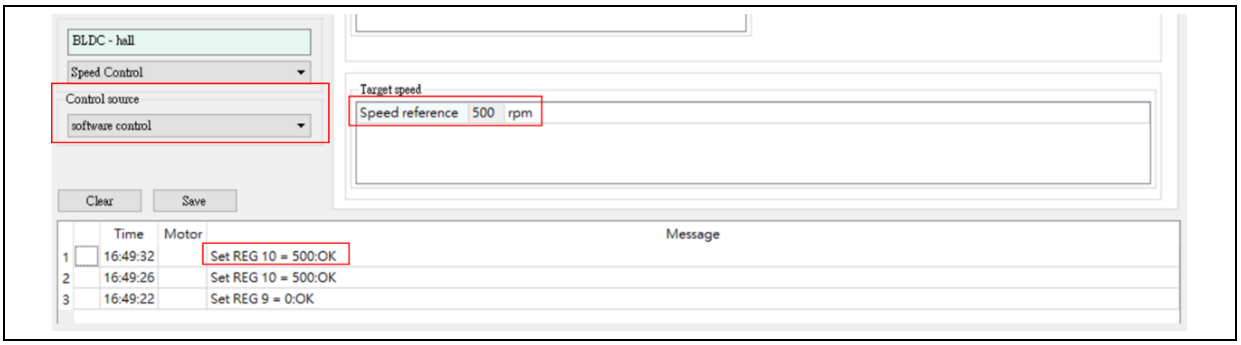
Figure 56. Torque control mode (external voltage control)



2) Select “Software control” as a control source

The software control mode is selected by default, as shown in Figure 57. In this mode, the user can adjust the motor speed and torque by changing the target speed/torque reference. The “message” area will show whether the changing operation is successful or not.

Figure 57. Set target speed in software control mode



4 Revision history

Table 19. Document revision history

Date	Version	Revision note
2024.05.24	2.0.0	Initial release.
2024.12.27	2.1.1	1. Added AT32F435/437 series and AT32L021 series support; 2. Added description of IAR environment and *icf file modification for Flash configuration; 3. Updated definitions, code and some descriptions.
2025.04.11	2.1.2	Added support for AT32F425 series and Field-Weakening (FW) function description
2025.10.09	2.1.3	Added new supported models, fixed flash configuration table and parameter updates

IMPORTANT NOTICE – PLEASE READ CAREFULLY

Purchasers are solely responsible for the selection and use of ARTERY's products and services; ARTERY assumes no liability for purchasers' selection or use of the products and the relevant services.

No license, express or implied, to any intellectual property right is granted by ARTERY herein regardless of the existence of any previous representation in any forms. If any part of this document involves third party's products or services, it does NOT imply that ARTERY authorizes the use of the third party's products or services, or permits any of the intellectual property, or guarantees any uses of the third party's products or services or intellectual property in any way.

Except as provided in ARTERY's terms and conditions of sale for such products, ARTERY disclaims any express or implied warranty, relating to use and/or sale of the products, including but not restricted to liability or warranties relating to merchantability, fitness for a particular purpose (based on the corresponding legal situation in any unjudicial districts), or infringement of any patent, copyright, or other intellectual property right.

ARTERY's products are not designed for the following purposes, and thus not intended for the following uses: (A) Applications that have specific requirements on safety, for example: life-support applications, active implant devices, or systems that have specific requirements on product function safety; (B) Aviation applications; (C) Aerospace applications or environment; (D) Weapons, and/or (E) Other applications that may cause injuries, deaths or property damages. Since ARTERY products are not intended for the above-mentioned purposes, if purchasers apply ARTERY products to these purposes, purchasers are solely responsible for any consequences or risks caused, even if any written notice is sent to ARTERY by purchasers; in addition, purchasers are solely responsible for the compliance with all statutory and regulatory requirements regarding these uses.

Any inconsistency of the sold ARTERY products with the statement and/or technical features specification described in this document will immediately cause the invalidity of any warranty granted by ARTERY products or services stated in this document by ARTERY, and ARTERY disclaims any responsibility in any form.